



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:**

***«Прогнозирование потребления электроэнергии на
основе данных IoT с использованием методов
машинного обучения»***

Студент ИУ9-81Б _____
(Группа)

(Подпись, дата) О.С. Александрова
(И.О. Фамилия)

Руководитель ВКР

(Подпись, дата) И.Э. Вишняков
(И.О. Фамилия)

Нормоконтролер

(Подпись, дата) С.М. Алексеев
(И.О. Фамилия)

2025 г.

АННОТАЦИЯ

Расчетно-пояснительная записка к выпускной квалификационной работе на тему «Прогнозирование потребления электроэнергии на основе данных IoT с использованием методов машинного обучения» состоит из 61 страниц, содержит 4 разделов, 21 листингов, 6 таблиц и 13 рисунков. Использовано 24 источников.

Цель работы — разработка системы прогнозирования потребления активной электроэнергии (A_+) на основе данных IoT, включающей модули анализа, предобработки и моделирования временных рядов с применением методов машинного обучения.

Актуальность исследования обусловлена необходимостью оптимизации ресурсов в системах умных электросетей. Разработан модуль анализа временных рядов энергопотребления для выявления сезонности и аномалий. Рассмотрены комбинации методов предобработки, включающих метод k -ближайших соседей, межквартильный размах, случайный лес, STL-разложение и нормализацию, для устранения пропусков, аномалий и приведения данных к единому масштабу. Реализованы модели прогнозирования ARIMA, N-BEATS и XGBoost, учитывающие характеристики временных рядов. Тестирование моделей выполнено на горизонтах до одного месяца с использованием временной кросс-валидации.

Результаты включают сравнительный анализ моделей, визуализацию прогнозов и результаты оценки их эффективности для данных с сезонной структурой.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 Обзор предметной области	9
1.1 Описание и характеристики данных	9
1.1.1 Структура данных	9
1.1.2 Анализ временных рядов	10
1.2 Проблемы и особенности обработки данных	13
1.2.1 Обработка пропусков	13
1.2.2 Обнаружение и обработка аномалий	15
1.2.3 Нормализация данных	17
1.2.4 Оценка качества предобработки данных	18
1.3 Обзор модели машинного обучения для прогнозирования	19
1.3.1 Задачи и требования прогнозирования	19
1.3.2 Модель ARIMA	20
1.3.3 Модель XGBoost	22
1.3.4 Модель N-BEATS	23
1.3.5 Оценка моделей прогнозирования	25
2 Проектирование приложения для прогнозирования потребления электроэнергии	28
2.1 Модуль анализа данных	29
2.2 Модуль предобработки данных	30
2.3 Модуль прогнозирования	31
3 Реализация приложения прогнозирования потребления электроэнергии	33
3.1 Реализация анализа данных	33
3.2 Реализация предобработки данных	34
3.2.1 Реализация заполнения пропусков	34
3.2.2 Реализация обработки аномалий	35
3.2.3 Реализация нормализации	37
3.3 Реализация модели ARIMA	38
3.4 Реализация модели XGBoost	39

3.5	Реализация модели N-BEATS	41
3.6	Реализация временного прогнозирования	42
4	Тестирование системы прогнозирования потребления элек- троэнергии	43
4.1	Анализ исходных данных	43
4.2	Тестирование предобработки данных	45
4.3	Анализ стационарности временного ряда	47
4.4	Тестирование моделей прогнозирования	49
4.4.1	Тестирование модели сезонной ARIMA	49
4.4.2	Тестирование модели XGBoost	51
4.4.3	Тестирование модели N-BEATS	52
4.5	Анализ результатов тестирования	54
	ЗАКЛЮЧЕНИЕ	56
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	57
	ПРИЛОЖЕНИЕ А	60

ВВЕДЕНИЕ

Рост энергопотребления, связанный с развитием технологий и цифровизации, подчёркивает необходимость точного прогнозирования для эффективного управления энергосистемами. Данные устройств Интернета вещей (IoT), таких как интеллектуальные счётчики, предоставляют основу для анализа потребления электроэнергии, поддерживая переход к умным энергосетям.

Актуальность исследования обусловлена несколькими аспектами. Точные прогнозы позволяют энергетическим компаниям оптимизировать процессы производства и распределения энергии, снижая эксплуатационные затраты и предотвращая перегрузки в сетях. Кроме того, интеграция данных от интеллектуальных устройств с аналитическими методами позволяет создавать адаптивные системы, способные реагировать на изменения условий [1]. Особенность задачи заключается в необходимости разработки методов, обеспечивающих качество предсказаний с учётом специфики данных, что требует создания подходящих инструментов для их обработки и анализа.

Работа с данными IoT осложняется наличием пропусков, аномалий и вариабельностью временных рядов, обусловленной сезонными, суточными факторами, а также индивидуальными особенностями потребителей. Таким образом, целью работы является разработка и исследование подходов к анализу, предобработке и прогнозированию потребления электроэнергии на основе данных Интернета вещей. Также необходимо разработать приложение прогнозирования потребления электроэнергии.

1 Обзор предметной области

1.1 Описание и характеристики данных

1.1.1 Структура данных

Используемый набор данных представляет собой временные ряды, собранные с умных электрических счетчиков, функционирующих как устройства Интернета вещей. Каждая запись в наборе данных включает следующие поля, характеризующие ключевые аспекты энергопотребления и генерации:

- `uuid`: уникальный идентификатор счетчика, позволяющий различать данные от разных объектов (домов или квартир). Это поле используется для группировки данных по отдельным потребителям и анализа индивидуальных особенностей потребления;
- `time_dt`: временная метка, указывающая дату и час измерения, для анализа временной структуры;
- A_+ : активная энергия, потребленная объектом за час (измеряется в Вт·ч). Этот параметр отражает фактическое потребление электроэнергии, связанное с работой бытовых приборов, освещением и другими нагрузками;
- A_- : активная энергия, возвращенная в сеть за час (измеряется в Вт·ч). Этот параметр актуален для объектов с установленными возобновляемыми источниками энергии, такими как солнечные панели, которые генерируют электроэнергию и передают излишки в сеть;
- R_+ : реактивная энергия, потребленная объектом за час (измеряется в ВАр·ч). Реактивная энергия связана с индуктивными или емкостными нагрузками (например, электродвигателями, трансформаторами) и не выполняет полезной работы, но влияет на качество работы энергосистемы. Высокие значения R_+ могут свидетельствовать о неэффективном использовании электроэнергии;
- R_- : реактивная энергия, возвращенная в сеть за час (измеряется в ВАр·ч). Этот параметр также связан с объектами, генерирующими элек-

троэнергию, и отражает передачу излишков реактивной энергии обратно в сеть.

Для задач прогнозирования в рамках данного исследования выбран параметр A_+ , поскольку, во-первых, он отражает фактическое потребление, что важно для планирования энергопроизводства. Во-вторых, предполагается, что A_+ обладает выраженными закономерностями, такими как суточные и сезонные колебания, которые могут быть смоделированы методами машинного обучения. В-третьих, в отличие от A_- , R_+ и R_- , которые часто равны нулю для объектов без генерации энергии, A_+ содержит ненулевые значения практически во всех записях, что упрощает обучение моделей.

Таким образом, выбор A_+ в качестве целевого параметра для прогнозирования позволяет сосредоточиться на ключевом аспекте потребления электроэнергии, обеспечивая максимальную практическую ценность результатов работы.

1.1.2 Анализ временных рядов

Анализ временных рядов параметра активной энергии A_+ , представляющего почасовые измерения потребления электроэнергии, позволяет выявить ключевые закономерности, необходимые для прогнозирования. Данные характеризуются сложной структурой, включающей сезонность, тренды, вариабельность и возможную нестационарность, что определяет подходы к их дальнейшей обработке и моделированию [2].

Основные характеристики проявляются через суточные и годовые циклы (например, вечерние пики потребления), долгосрочные изменения (связанные с подключением новых устройств), краткосрочные колебания (из-за нерегулярного использования или праздников) и нестационарность (изменение статистических свойств из-за трендов или сезонности). На графике временного ряда эти особенности отображаются так: повторяющиеся шаблоны указывают на сезонность, постепенные сдвиги — на тренды, стабильные участки — на равномерное потребление, резкие пики или провалы — на аномалии (например, сбои счетчиков), а чередование роста и падения — на вариабельность.

Для выделения этих компонентов применяется декомпозиция временных рядов, например, STL-разложение (Seasonal-Trend decomposition using LOESS)

[3]. Аддитивная модель разложения представлена как:

$$y_t = T_t + S_t + R_t \quad (1)$$

где T_t — тренд, S_t — сезонная компонента, R_t — остатки. При зависимости амплитуды сезонности от уровня ряда используется мультипликативная модель:

$$y_t = T_t \cdot S_t \cdot R_t.$$

В STL-разложении тренд и сезонность оцениваются с помощью локальной регрессии (LOESS):

$$\hat{y}_t = \sum_{i \in N_t} w_i y_i,$$

где веса w_i задаются функцией:

$$W(u) = (1 - |u|^3)^3, \quad u = \frac{|t_i - t|}{\Delta},$$

где t_i — момент времени наблюдения, t — текущий момент, Δ — половина ширины окна сглаживания, определяющая диапазон учитываемых точек. Параметр u — нормированное расстояние между t_i и t . Ширина окна Δ балансирует между сглаживанием и сохранением локальных особенностей: большое Δ даёт более плавный тренд, малое — чувствительность к изменениям. STL устойчиво к пропускам (интерполяция в процессе сглаживания) и аномалиям (снижение влияния выбросов через веса), что важно для данных IoT.

Средние значения компонент STL несут важную информацию. Среднее значение тренда (T_t) показывает общий уровень потребления, например, устойчивый рост из-за увеличения числа устройств. Среднее значение сезонной компоненты (S_t), близкое к нулю, указывает на сбалансированные циклические колебания, а ненулевое — на систематическое смещение, например, более высокое потребление в определённые часы. Остатки (R_t) отражают случайные или аномальные колебания: их среднее близкое к нулю и малая амплитуда свидетельствуют о качественной декомпозиции, тогда как значительные отклонения указывают на выбросы или пропущенные закономерности.

Для некоторых моделей машинного обучения требуется стационарность ряда, когда его статистические характеристики (среднее, дисперсия, автокорреляция) постоянны во времени. Нестационарность, вызванная трендами или сезонностью, затрудняет прогнозирование [2]. Тест Дики-Фуллера (англ. Augmented Dickey-Fuller, ADF) [4] проверяет наличие единичного корня, указывающего на нестационарность, через модель:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \dots + \delta_{p-1} \Delta y_{t-p+1} + \varepsilon_t,$$

где $\Delta y_t = y_t - y_{t-1}$ — первая разность, α — константа, βt — линейный тренд, γ — коэффициент единичного корня, δ_i — коэффициенты лагов, ε_t — остатки. Нулевая гипотеза ($H_0: \gamma = 0$) предполагает нестационарность, альтернативная ($H_1: \gamma < 0$) — стационарность. p-value показывает вероятность получения статистики при верной H_0 : если оно меньше 0,05, ряд стационарен. При нестационарности применяют дифференцирование:

$$\Delta y_t = y_t - y_{t-1},$$

или сезонное дифференцирование:

$$y_t - y_{t-24},$$

для устранения суточной сезонности. Визуально стационарность можно оценить на графике: стационарный ряд колеблется вокруг постоянного среднего без выраженных трендов или сезонных шаблонов.

В дополнение, анализ гистограмм распределения A_+ выявляет структуру данных. Правосторонняя асимметрия (хвост вправо) указывает на преобладание низких или средних значений с редкими пиками, например, в часы повышенного спроса, а левосторонняя — на частое высокое потребление. Гистограммы также помогают обнаружить выбросы как редкие значения в длинных хвостах. Асимметрия усложняет анализ, поскольку многие статистические методы и модели предполагают нормальное распределение данных. Для устранения этого эффекта и нормализации распределения применяется логарифмическое преобразование,

которое сглаживает асимметрию и делает данные более пригодными для последующей обработки.

Важно отметить, что выбор периода анализа определяет, какие закономерности можно выявить. Суточный период детализирует краткосрочные колебания, такие как пики в вечерние часы. Недельный период показывает различия между буднями и выходными. Период за месяц наиболее информативен для выявления сезонных и трендовых шаблонов, таких как рост потребления в определённые сезоны. Визуально суточные пики заметны на графике как регулярные всплески каждые 24 часа, недельные — как изменения между днями, а за месяц — как более плавные изменения, видимые на графике STL-декомпозиции.

Таким образом, анализ временных рядов, сочетающий визуальные и статистические методы, позволяет выявить закономерности и аномалии в данных A_+ , подготавливая их для предобработки и последующего прогнозирования.

1.2 Проблемы и особенности обработки данных

Обработка данных, собранных с умных электрических счётчиков, осложняется потенциальными пропусками, аномалиями и высокой вариабельностью параметра A_+ , которые могут возникать во временных рядах и снижать точность прогнозирования потребления электроэнергии [2]. В этом разделе рассматриваются указанные особенности и предлагаются методы, позволяющие их преодолевать, позволяющие выбрать оптимальную стратегию подготовки данных для моделей прогнозирования.

1.2.1 Обработка пропусков

Пропуски в данных A_+ , вызванные сбоями оборудования или передачи данных, нарушают целостность временных рядов, искажая сезонность и тренды [2]. Под пропуском понимается отсутствие значения за определённый час при почасовом сборе данных. Непрерывность ряда необходима для моделей машинного обучения, поэтому рассматриваются методы ее восстановления.

Линейная интерполяция [5] вычисляет пропущенное значение y_t по формуле:

$$y_t = y_{t-k} + \frac{(y_{t+m} - y_{t-k}) \cdot (t - (t - k))}{m + k} \quad (2)$$

где y_{t-k} , y_{t+m} — ближайшие известные значения. Метод прост и подходит для коротких пропусков, но игнорирует суточную сезонность, что может исказить циклические шаблоны.

Сезонная интерполяция [6] основана на усреднении значений за предыдущие сутки:

$$y_t = \frac{1}{n} \sum_{i=1}^n y_{t-24i},$$

где n — число суточных циклов. Метод можно использовать при выраженной суточной сезонности, но он ненадёжен для длительных пропусков или слабой сезонности, так как игнорирует локальные тренды и может занижать дисперсию. Выбор n зависит от стабильности сезонности, проверяемой автокорреляцией на лаге 24 часа.

Метод KNN-восстановления [7] заполняет пропуски на основе k -ближайших по времени значений, используя расстояние:

$$d(t_i, t_j) = |t_i - t_j|.$$

Значение y_t вычисляется как:

$$y_t = \sum_{i \in N_k} w_i y_i, \quad w_i = \frac{1/d(t, t_i)}{\sum_{j \in N_k} 1/d(t, t_j)},$$

где N_k — множество соседей, w_i — веса. Метод учитывает локальные зависимости, сохраняя дисперсию, и подходит для длительных пропусков, но вычислительно сложен и менее точен при высокой плотности пропусков.

Дисперсия ряда, заданная как:

$$\sigma_y^2 = \frac{1}{n} \sum_{t=1}^n (y_t - \bar{y})^2,$$

где $\bar{y}$ — среднее значение A_+ , отражает вариабельность потребления электроэнергии[2]. Сохранение дисперсии при восстановлении пропусков важно, так как её искажение ухудшает качество моделей, чувствительных к сезонности. Линейная интерполяция может увеличивать дисперсию, сезонная — занижать, а KNN-восстановление лучше сохраняет статистические свойства.

1.2.2 Обнаружение и обработка аномалий

Аномалии в данных A_+ , представляющие значения, отклоняющиеся от типичного поведения временного ряда, могут исказить результаты прогнозирования [8]. Такие отклонения возникают из-за сбоев оборудования, ошибок измерений или пиков потребления, что затрудняет их классификацию как ошибок или достоверных значений. Обнаружение и обработка аномалий необходимы для подготовки данных к моделированию, сохраняя при этом сезонность и тренды, определённые на этапе анализа временных рядов. Ниже рассматриваются методы, адаптированные к структуре данных и характеру отклонений.

Первым методом является метод межквартильного размаха (англ. IQR) [9], который выявляет аномалии на основе квартилей ряда. Он определяется следующим образом:

$$Q1 = Q_{0.25}, \quad Q3 = Q_{0.75}, \quad IQR = Q3 - Q1,$$

где значения, выходящие за границы $[Q1 - c \cdot IQR, Q3 + c \cdot IQR]$, классифицируются как аномалии. Этот метод прост в реализации и не требует предположений о распределении данных, однако он не учитывает суточную сезонность, что может привести к ошибочной идентификации естественных пиков потребления. Параметр c регулирует чувствительность подхода, а его выбор зависит от ожидаемой доли аномалий и вариабельности ряда, которые оцениваются с помощью гистограммы распределения A_+ .

Второй метод — STL-разложение [3] — основан на декомпозиции временного ряда в соответствии с аддитивной моделью (1). Аномалии здесь определяются через остатки, превышающие заданный порог:

$$|R_t| > k \cdot \sigma_R, \tag{3}$$

где $\sigma_R = \sqrt{\frac{1}{n} \sum_{i=1}^n (R_i - \bar{R})^2}$ — стандартное отклонение остатков, вычисляемое как квадратный корень из дисперсии остаточной компоненты, k — порог. Метод учитывает циклические шаблоны, что делает его подходящим для рядов с выраженной сезонностью, но требует устойчивости к пропускам и достаточного объема данных. Период сезонности p устанавливается в зависимости от структуры данных

(для почасовых данных — 24 часа), а порог k определяется на основе амплитуды остатков, оцениваемой визуально или статистически. Ограничением метода является его неспособность выявлять аномалии, связанные с трендами, поскольку сосредоточение на сезонных и остаточных компонентах исключает анализ долгосрочных изменений, которые могут затруднять обнаружение аномальных значений в общей тенденции.

Третий подход — метод Random Forest (RF) [10] — использует ансамбль деревьев решений для регрессионного прогнозирования значений y_t . Аномалии определяются через отклонение между наблюдаемыми и предсказанными значениями:

$$|y_t - \hat{y}_t| > \theta \cdot \text{MAD},$$

где $\hat{y}_t$ — предсказанное значение, а $\text{MAD} = \text{median}(|y_t - \hat{y}_t|)$ — медиана абсолютных отклонений, θ — значение вычисляемого порога. Метод строит множество деревьев, каждое из которых обучается на случайной подвыборке данных и признаков, а итоговое предсказание формируется путём усреднения их результатов. Это обеспечивает устойчивость к шуму и снижает риск переобучения.

Применение метода Random Forest к временным рядам, таким как A_+ , предполагает формирование специальных признаков, отражающих динамику потребления. К ним относятся лаги (например, y_{t-1} , y_{t-2}), час суток, скользящие статистики (среднее, медиана) и календарные факторы (будни/выходные). Эти признаки учитывают сложные зависимости, включая суточную сезонность, краткосрочные тренды и нерегулярные шаблоны, что подтверждается результатами декомпозиции и анализа автокорреляции.

Настройка данного метода требует определения ключевых параметров, влияющих на его производительность. Порог θ регулирует чувствительность к аномалиям и устанавливается на основе анализа распределения отклонений $|y_t - \hat{y}_t|$ с использованием гистограмм или квантилей. Число деревьев в ансамбле подбирается с учётом объёма данных и их вариабельности, обеспечивая баланс между стабильностью предсказаний и вычислительными затратами. Однако метод характеризуется значительной вычислительной сложностью, особенно при обработке больших объёмов информации, а его эффективность существенно зависит от качества признаков: недостаточный их набор снижает точность, тогда как избыточное количество может привести к переобучению.

Обработка аномалий может включать замену с помощью линейной интерполяции, описанной формулой (2), или удаление с последующей интерполяцией. Замена сохраняет целостность ряда, но может сглаживать пики, что влияет на анализ нагрузок. Выбор подхода определяется задачами моделирования и природой аномалий.

Таким образом, выбор метода зависит от свойств данных: выраженности сезонности, объема и доли аномалий. STL предпочтителен для циклических рядов, RF — для нерегулярных, а IQR — для быстрого анализа. Неправильный выбор может привести к исключению реальных пиков, влияя на качество моделей машинного обучения, чувствительных к структуре данных.

1.2.3 Нормализация данных

Нормализация данных A_+ устраняет высокую вариативность, обусловленную сезонностью, трендами и индивидуальными особенностями потребителей, обеспечивая единообразие входных данных для моделей машинного обучения [11]. Рассмотрим методы, учитывающие специфику данных об употреблении электроэнергии.

Линейное масштабирование [11] преобразует данные в диапазон $[0, 1]$ по формуле:

$$y'_t = \frac{y_t - y_{\min}}{y_{\max} - y_{\min}},$$

где $y_{\min}$, $y_{\max}$ — минимальное и максимальное значения ряда, ϵ — малая константа для численной стабильности. Метод сохраняет форму распределения и подходит для моделей, чувствительных к масштабу. Однако он уязвим к выбросам, сжимающим значения в узкий интервал.

Логарифмическое преобразование [2] задаётся как:

$$y'_t = \ln(y_t + c),$$

где c — константа. Метод уменьшает асимметрию и стабилизирует вариативность, что подходит для рядов с пиками потребления. Параметр c выбирается с учётом минимальных значений ряда. Преобразование сглаживает пики, улучшая устойчивость моделей, но усложняет интерпретацию сезонности из-за нелинейности.

Денормализация восстанавливает исходный масштаб для интерпретации результатов. Для линейного масштабирования используется:

$$y_t = y'_t \cdot (y_{\max} - y_{\min}) + y_{\min},$$

для логарифмического преобразования:

$$y_t = \exp(y'_t) - c.$$

При выборе метода нормализации учитываются характеристики данных: асимметрия распределения и выраженность сезонности. Линейное масштабирование подходит для однородных рядов, логарифмическое преобразование — для асимметричных.

1.2.4 Оценка качества предобработки данных

Предобработка данных A_+ , охватывающая обработку пропусков, аномалий и нормализацию, подготавливает временные ряды к моделированию, минимизируя искажения, которые могут повлиять на точность прогнозов. Качество этого процесса оценивается через набор метрик, учитывающих вариабельность и сезонность данных об употреблении электроэнергии.

Общие метрики, применимые ко всем этапам, включают следующие показатели, вычисляемые для каждого этапа обработки:

- среднее — характеризует базовый уровень ряда и помогает оценить общий сдвиг значений после предобработки:

$$\mu = \frac{1}{n} \sum_{t=1}^n y_t \quad (4)$$

- стандартное отклонение — отражает вариабельность данных, позволяя выявить изменения в разбросе значений, вызванные сглаживанием или усилением колебаний:

$$\sigma = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \mu)^2} \quad (5)$$

- автокорреляция на суточном лаге ($ACF(24)$) — измеряет сезонную зависимость, указывая на сохранность или утрату циклических шаблонов:

$$ACF(k) = \frac{\sum_{t=k+1}^n (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^n (y_t - \bar{y})^2}, \quad (6)$$

где $\bar{y}$ — среднее ряда [12].

Сравнение этих метрик с исходными значениями позволяет оценить сохранность структуры ряда. Также, вычисление минимального и максимального значения ряда помогают выявить границы распределения и обнаружить экстремальные значения, сохранившиеся после обработки.

Комплексный анализ метрик в сочетании с визуальными представлениями (например, гистограммами распределений) позволяет выбрать методы, балансирующие устранение ошибок и сохранение информативности данных для последующего прогнозирования.

1.3 Обзор модели машинного обучения для прогнозирования

Прогнозирование потребления электроэнергии (A_+) является ключевой задачей для оптимизации управления энергосистемами. Предобработка данных подготавливает A_+ для предсказания будущих значений на горизонтах от 7 дней до месяца.

1.3.1 Задачи и требования прогнозирования

Прогнозирование временных рядов заключается в предсказании будущих значений потребления электрической энергии (A_+) на основе исторических данных $\{y_1, y_2, \dots, y_t\}$, где h — горизонт прогнозирования. В контексте данных A_+ прогнозирование решает следующие задачи:

- оптимизация распределения ресурсов: прогнозы позволяют повысить качество планирования генерации и распределения электроэнергии;
- улучшение спроса: прогнозы помогают поддерживать меры по снижению пиковых нагрузок;
- предотвращение перегрузок: предсказание всплесков потребления минимизирует риск сбоев в энергосистеме.

Для достижения этих целей модели должны учитывать следующие требования:

- моделировать сезонность (суточную и годовую) и тренды;
- учитывать индивидуальные шаблоны пользователей (uuid);
- быть устойчивыми к нерегулярным данным и пропускам;
- обеспечивать интерпретируемость прогнозов для поддержки решений в энергетике.

Таким образом, модель должна поддерживать краткосрочные и долгосрочные горизонты, обрабатывая сложные временные зависимости и нерегулярности IoT-данных.

1.3.2 Модель ARIMA

Модель ARIMA (AutoRegressive Integrated Moving Average) — классический метод прогнозирования временных рядов, широко применяемый для анализа данных с линейными и сезонными закономерностями [2,6]. Она выбрана для прогнозирования данных A_+ благодаря своей интерпретируемости, устойчивости при достаточном объёме данных и способности моделировать циклические шаблоны потребления электроэнергии.

ARIMA объединяет три компоненты: авторегрессию (AR), дифференцирование (I) для достижения стационарности и скользящее среднее (MA). Для данных A_+ с выраженной сезонностью используется сезонная ARIMA, обозначаемая как $(p,d,q) \times (P,D,Q)_s$, где s — сезонный период (например, 24 для почасовых данных). Уравнение модели:

$$\Phi(B^s)\phi(B)(1-B)^d(1-B^s)^D y_t = \Theta(B^s)\theta(B)\epsilon_t,$$

где B — оператор лага ($By_t = y_{t-1}$), ϵ_t — белый шум, $\phi(B)$, $\Phi(B^s)$ — авторегрессионные полиномы порядков p и P , а $\theta(B)$, $\Theta(B^s)$ — полиномы скользящего среднего порядков q и Q .

Стационарность ряда A_+ достигается дифференцированием:

$$\Delta^d y_t = (1-B)^d y_t, \quad \Delta_s^D y_t = (1-B^s)^D y_t,$$

где d устраняет тренды, а D — сезонные колебания. Параметр d и D подбираются с помощью теста Дики–Фуллера [4], проверяющего наличие единичных корней. Порядки p и P определяются по значимым лагам частичной автокорреляционной функции (Partial Autocorrelation Function, PACF), которая оценивает прямую зависимость между y_t и y_{t-k} , исключая влияние промежуточных лагов. Формально PACF определяется как:

$$\text{PACF}(k) = \frac{\text{Cov}(y_t - \hat{y}_t, y_{t-k} - \hat{y}_{t-k})}{\sqrt{\text{Var}(y_t - \hat{y}_t)\text{Var}(y_{t-k} - \hat{y}_{t-k})}}, \quad (7)$$

где:

- $\hat{y}_t$ и $\hat{y}_{t-k}$ — прогнозы y_t и y_{t-k} на основе линейной регрессии по промежуточным лагам $y_{t-1}, \dots, y_{t-k+1}$;
- Cov — ковариация;
- Var — дисперсия.

Порядки q и Q выбираются по автокорреляционной функции (6), измеряющей корреляцию между y_t и y_{t-k} .

Для выбора оптимальной конфигурации параметров $(p, d, q) \times (P, D, Q)_s$ используется критерий Акаике (Akaike information criterion, AIC), который балансирует качество подгонки модели и её сложность [13]. AIC определяется как:

$$\text{AIC} = 2k - 2 \ln(L), \quad (8)$$

где k — число параметров модели, а L — значение функции правдоподобия. Модель с минимальным значением AIC предпочтительна, так как она обеспечивает наилучшее соотношение между точностью прогноза и переобучением. Применение AIC позволяет автоматизировать процесс выбора параметров, повышая эффективность настройки модели для данных A_+ .

Модель интерпретируема, что позволяет анализировать влияние сезонных и трендовых компонент, а её умеренные вычислительные требования делают её подходящей для больших объёмов данных. Однако ARIMA ограничена в моделировании нелинейных зависимостей и сложных нерегулярных шаблонов, которые могут возникать в данных с разнородными потребителями, что подтверждается исследованиями [2]. Для таких случаев можно использовать методы машинного обучения, такие как рекуррентные нейронные сети или ансамблевые алгоритмы.

1.3.3 Модель XGBoost

XGBoost (eXtreme Gradient Boosting) — метод машинного обучения, основанный на градиентном бустинге деревьев решений, эффективный для прогнозирования временных рядов с нелинейными зависимостями[14,15]. Модель выбрана для данных A_+ благодаря способности моделировать сложные шаблоны потребления электроэнергии, такие как нерегулярные пики и сезонные колебания, а также её устойчивости к шуму и высокой точности при правильной настройке.

Модель XGBoost строит ансамбль деревьев решений, где каждое дерево корректирует ошибки предыдущих, формируя итоговый прогноз как их сумму:

$$\hat{y}_{t+h} = \sum_{m=1}^M f_m(\mathbf{x}_t),$$

где $\mathbf{x}_t \in \mathbb{R}^d$ — вектор признаков, f_m — предсказание m -го дерева, M — число деревьев. Признаки для A_+ включают лаги ряда ($y_{t-1}, y_{t-24}, y_{t-48}$), сезонные индикаторы (час дня, день недели, месяц) и характеристики пользователей. Эти признаки отражают суточную и годовую сезонность, а также индивидуальные шаблоны потребления.

Модель оптимизирует функцию потерь, сочетая ошибку предсказания и регуляризацию:

$$L = \sum_{t=1}^n \ell(y_t, \hat{y}_t) + \sum_{m=1}^M \Omega(f_m),$$

где $\ell(y_t, \hat{y}_t)$ — функция потерь (например, среднеквадратичная ошибка/MSE), а компонента $\Omega(f_m) = \gamma T_m + \frac{1}{2} \lambda \sum_{j=1}^{T_m} w_{m,j}^2$ предотвращает переобучение, контролируя число листьев T_m и веса листьев $w_{m,j}$. Оптимизация выполняется итеративно, добавляя новые деревья, которые минимизируют остатки:

$$\hat{y}_t^{(m)} = \hat{y}_t^{(m-1)} + \eta f_m(\mathbf{x}_t),$$

где $\eta \in (0, 1]$ — скорость обучения.

Ключевые гиперпараметры включают:

- число деревьев M — определяет размер ансамбля, балансируя между точностью и вычислительной сложностью;

- η (скорость обучения) — контролирует вклад каждого дерева, меньшие значения улучшают устойчивость, но требуют больше итераций;
- γ и λ (регуляризация) — ограничивают сложность деревьев, предотвращая переобучение;
- максимальная глубина деревьев — влияет на способность моделировать сложные зависимости, но увеличивает риск переобучения;
- подвыборка признаков и данных — повышает устойчивость и ускоряет обучение.

Модель XGBoost выбрана для A_+ , так как данные содержат нелинейные зависимости, нерегулярные пики и разнородные шаблоны потребления, которые сложно моделировать классическими методами, такими как ARIMA. Модель использует признаки, сформированные на основе сезонности и лагов, однако требует тщательной настройки гиперпараметров и качественных признаков, а также может быть вычислительно затратным при больших ансамблях. Для данных с сильной сезонностью и линейными трендами классические модели могут быть предпочтительнее.

1.3.4 Модель N-BEATS

Модель N-BEATS (Neural Basis Expansion Analysis for Time Series) была выбрана для прогнозирования данных энергопотребления A_+ благодаря её высокой эффективности на сезонных временных рядах и способности обеспечивать интерпретируемые результаты [16,17].

Модель N-BEATS представляет собой глубокую нейронную сеть, которая формирует прогноз как нелинейную функцию от нормализованных входных данных:

$$\hat{y}_{t+h} = f(\mathbf{x}_t; \theta),$$

где $\mathbf{x}_t = [y_{t-L}, \dots, y_t] \in \mathbb{R}^L$ — вектор исторических значений длиной L (период ретроспективы, например, $L = 24$ для суточной сезонности A_+), h — горизонт прогноза, а θ — обучаемые параметры сети. Архитектура состоит из S стеков, каждый из которых содержит B базовых блоков. Каждый стек раскладывает входной ряд на трендовые и сезонные компоненты, а итоговый прогноз формируется

как сумма предсказаний всех стеков:

$$\hat{y}_{t+h} = \sum_{s=1}^S g_s(\mathbf{x}_t),$$

где $g_s : \mathbb{R}^L \rightarrow \mathbb{R}^h$ — выходная функция s -го стека, отображающая входной вектор $\mathbf{x}_t$ в вектор прогноза длиной h .

На рисунке 1 представлена архитектура модели N-BEATS, иллюстрирующая обработку данных A_+ через иерархию блоков и стека с разложением на трендовые и сезонные компоненты. Каждый базовый блок b в стеке s принимает входной вектор $\mathbf{x}_{s,b}$ (для первого блока в стеке $\mathbf{x}_{s,1} = \mathbf{x}_t$) и состоит из последовательности полносвязных слоев (fully connected layers, FC) с функцией активации ReLU. Эти слои формируют скрытые представления $\mathbf{h}_{s,b,1}, \dots, \mathbf{h}_{s,b,4}$, которые затем преобразуются в коэффициенты тренда $\theta_{s,b}^b$ и сезонности $\theta_{s,b}^f$ через линейные слои, как показано на рисунке 1.

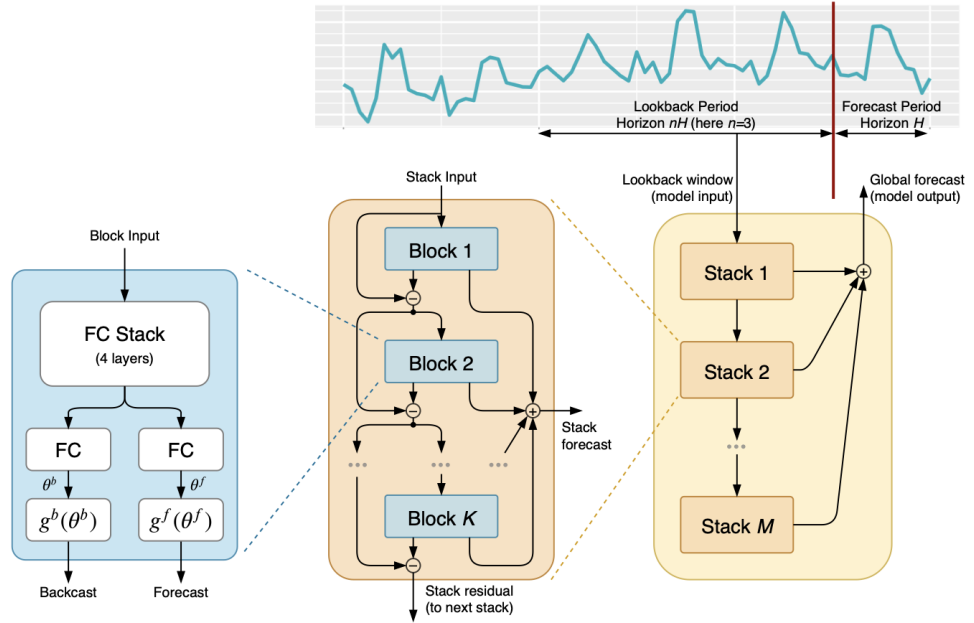


Рисунок 1 — Архитектура модели N-BEATS [16]

$$\mathbf{h}_{s,b,1} = \text{ReLU}(\text{FC}_1(\mathbf{x}_{s,b})), \quad \mathbf{h}_{s,b,\ell} = \text{ReLU}(\text{FC}_\ell(\mathbf{h}_{s,b,\ell-1})), \quad \ell = 2, 3, 4,$$

$$\theta_{s,b}^b = \text{LINEAR}^b(\mathbf{h}_{s,b,4}), \quad \theta_{s,b}^f = \text{LINEAR}^f(\mathbf{h}_{s,b,4}),$$

где LINEAR^b и LINEAR^f — линейные слои, отображающие скрытое представление в коэффициенты. Эти коэффициенты применяются к базисным функциям для формирования прогноза вперед $\mathbf{y}_{s,b} \in \mathbb{R}^h$ и остатка $\mathbf{x}_{s,b+1} \in \mathbb{R}^L$, что также отражено на схеме архитектуры:

$$\mathbf{y}_{s,b} = \sum_{i=1}^{d_f} \theta_{s,b,i}^f \mathbf{v}_i^f, \quad \mathbf{x}_{s,b+1} = \mathbf{x}_{s,b} - \sum_{i=1}^{d_b} \theta_{s,b,i}^b \mathbf{v}_i^b,$$

где $\mathbf{v}_i^f \in \mathbb{R}^h$ и $\mathbf{v}_i^b \in \mathbb{R}^L$ — базисные функции, d_f и d_b — их размерности. В интерпретируемой версии модели N-BEATS базисные функции задаются явно:

- для сезонности: $v_{i,j}^f = \sin\left(2\pi i \frac{j}{s}\right)$ или $v_{i,j}^f = \cos\left(2\pi i \frac{j}{s}\right)$, где $j = 1, \dots, h$, s — период (например, $s = 24$);
- для тренда: $v_{i,j}^b = \left(\frac{j}{L}\right)^{i-1}$, где $j = 1, \dots, L$, $i = 1, \dots, d_b$.

Выход стека $g_s(\mathbf{x}_t) \sum_{b=1}^B \mathbf{y}_{s,b}$, а остаток, вычисляемый как $\mathbf{x}_{s+1,1} = \mathbf{x}_{s,B+1}$, передается в следующий стек, что наглядно демонстрирует иерархическую структуру, представленную на рисунке 1.

Процесс обучения основан на иерархической агрегации и остаточном подходе: каждый блок предсказывает компоненты, а остаток передается следующему блоку. Обучение минимизирует среднеквадратичную ошибку на тренировочных данных:

$$L = \frac{1}{N} \sum_{t=1}^N \sum_{h=1}^H (y_{t+h} - \hat{y}_{t+h})^2,$$

где N — число тренировочных примеров, H — максимальный горизонт прогноза.

Для данных наподобие временных рядов с A_+ , модель N-BEATS захватывает суточные и годовые сезонности, а также индивидуальные шаблоны пользователей (uuid), не требуя ручного создания признаков, что подтверждено результатами экспериментов [16,18].

1.3.5 Оценка моделей прогнозирования

В рамках работы предполагается обучение рассмотренных моделей (ARIMA, XGBoost и N-BEATS) с временной кросс-валидацией и последующая оценка их качества с помощью метрик:

— средней абсолютной ошибки (Mean Absolute Error, MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (9)$$

— среднеквадратичной ошибки (Root Mean Square Error, RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (10)$$

где y_i — фактическое значение, $\hat{y}_i$ — предсказанное, n — число наблюдений. MAE измеряет среднюю абсолютную ошибку прогноза. Низкое значение MAE указывает на стабильность предсказаний модели. RMSE, в свою очередь, учитывает квадраты ошибок, что делает её чувствительной к значительным отклонениям, таким как пики потребления в A_+ . Разница между MAE и RMSE позволяет оценить, насколько модель справляется с экстремальными значениями, а их совместный анализ выявляет влияние предобработки: например, недостаточная коррекция аномалий может увеличить RMSE, указывая на проблемы с пиковыми значениями.

Данные A_+ разделяются на обучающую и тестовую выборки с сохранением хронологического порядка, чтобы исключить утечку информации из будущего в прошлое. Обучающая выборка охватывает более длительный период, достаточный для учёта сезонных закономерностей (например, годовой цикл), а тестовая — более короткий период для проверки обобщающей способности моделей.

Для обучения применяется временная кросс-валидация, которая сохраняет хронологическую последовательность данных и имитирует реальные условия прогнозирования [2]. Этот подход основан на принципе расширяющегося окна: модель последовательно обучается на нарастающем наборе исторических данных, начиная с начального периода, и проверяется на последующих отрезках времени, представляющих валидационные горизонты. Такой процесс повторяется для различных горизонтов, что позволяет оценить способность модели адаптироваться к сезонным изменениям и долгосрочным трендам. Временная кросс-валидация выявляет устойчивость моделей к вариациям во времени, их обобщающую способность и влияние качества предобработки на точность прогнозов.

В итоге, результатом оценки качества модели является:

- сравнение моделей прогнозирования по их способности моделировать сезонность, нелинейные зависимости и разнообразные шаблоны потребления;
- оценка влияния качества предобработки для выявления оптимальных подходов к подготовке данных;
- получения прогноза с учётом суточных, недельных закономерностей.

2 Проектирование приложения для прогнозирования потребления электроэнергии

Проектируемое приложение предназначено для анализа и прогнозирования потребления активной электроэнергии (A_+). Основная задача — разработка инструмента для обработки временных рядов и построение моделей прогнозирования, адаптированных к особенностям потребления электроэнергии. Для достижения этой цели приложение должно предоставлять возможности для изучения данных, их подготовки и применения моделей машинного обучения для предсказания целевого параметра.

С учетом особенностей данных приложение проектируется так, чтобы:

- анализировать данные, выявляя тренды, сезонные колебания и статистические показатели;
- обрабатывать данные, устраняя пропуски, аномалии и нормализуя значения с использованием настраиваемых методов;
- выполнять прогнозирование с помощью моделей ARIMA, XGBoost и N-BEATS, оптимизируя их гиперпараметры;
- тестировать прогнозы через временную кросс-валидацию и собирать метрики для оценки качества;
- обеспечивать модульную структуру для удобства разработки и поддержки.

Для настройки параметров используется конфигурационный файл в формате YAML. Параметры хранятся в виде пар "ключ-значение" с иерархической структурой, включая вложенные словари. Основные секции: `data` определяет пути к используемым данным, `preprocessing` задает методы обработки пропусков, аномалий и нормализации, `forecasting` настраивает модель прогнозирования, горизонты и гиперпараметры. Конфигурация загружается при запуске и передается всем модулям.

Проектируемое решение состоит из четырех основных модулей: анализ данных, предобработка, прогнозирование и оценка качества. Каждый модуль разработан таким образом, чтобы пользователь мог настраивать методы и параметры. Общая структура взаимодействия компонентов программного решения представлена на рисунке 2.



Рисунок 2 — Структурная схема проектируемого программного решения

2.1 Модуль анализа данных

Первым этапом обработки временных рядов является их анализ для выявления характеристик, закономерностей и особенностей, необходимых для подготовки данных и прогнозирования.

Модуль анализа данных обеспечивает для каждой группы (*uuid*) вычисление следующих характеристик:

- среднего значения и стандартного отклонения;
- минимального и максимального значения для выявления пиков и аномалий;
- проверки стационарности с использованием теста Дики-Фуллера;
- автокорреляции (ACF) и частичной автокорреляции (PACF).

Модуль должен поддерживать декомпозицию временных рядов (STL-разложение) для выделения тренда, сезонности и остатков, а также визуализацию данных, включая:

- графики временных рядов для оценки динамики;
- гистограммы распределения значений (A) для анализа их структуры;
- графики автокорреляции и декомпозиции для выявления закономерностей и шумов.

2.2 Модуль предобработки данных

Точность прогнозов потребления электроэнергии напрямую зависит от подготовки временных рядов. Проектируемый модуль предобработки данных IoT, включая A_+ , решает следующие задачи:

- заполнение пропусков с помощью линейной, сезонной, KNN-восстановления;
- выявление и устранение аномалий методами IQR, STL-разложения, построения случайного леса;
- нормализация (масштабирование к интервалу $[0; 1]$, логарифмическое преобразование).

Центральным компонентом модуля является класс DataPrep, который координирует процесс обработки, взаимодействуя через интерфейс Preprocessing с подчиненными компонентами, реализующими отдельные функции предобработки. Параметры методов и порогов берутся из конфигурационного файла. Для контроля качества предобработки вычисляются стандартное отклонение и полнота обработки (доля пропусков и аномалий). Схема взаимодействия компонентов изображена на рисунке 3.

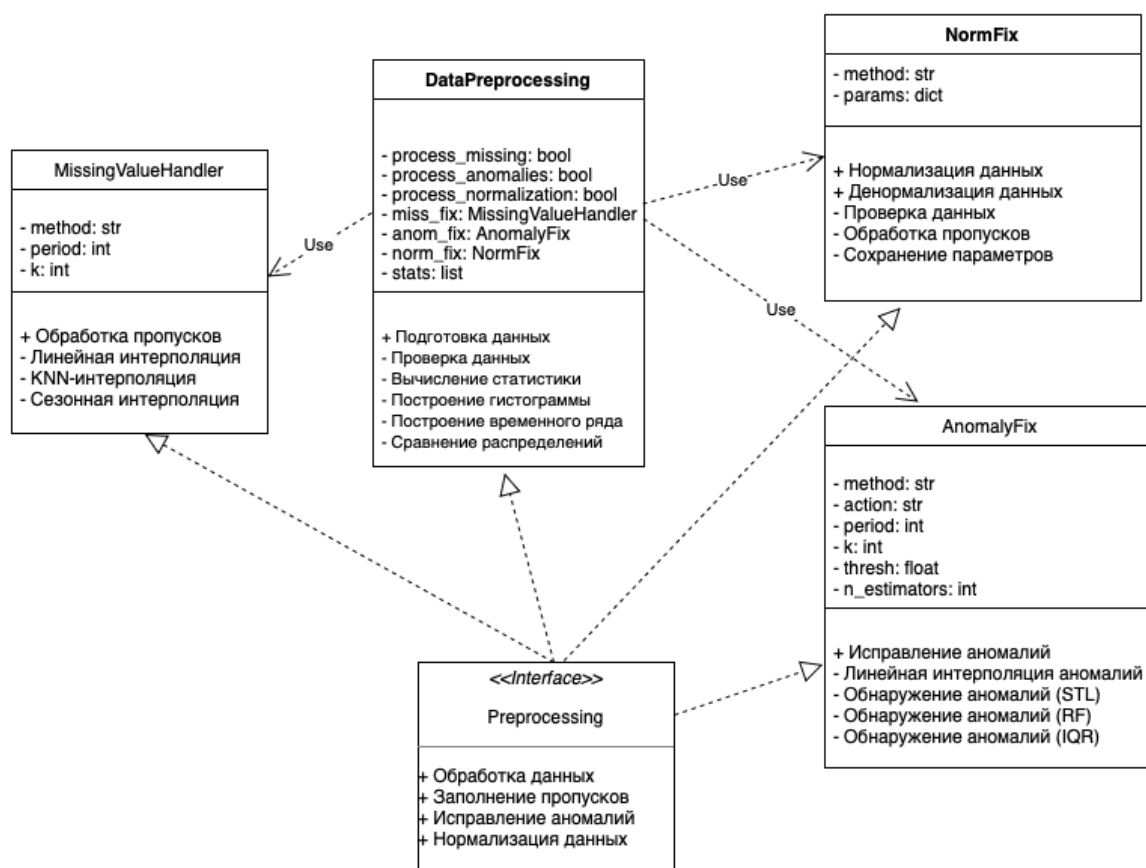


Рисунок 3 — Схема взаимодействия компонентов модулей предобработки данных

2.3 Модуль прогнозирования

Модуль прогнозирования разрабатывается для предсказания потребления активной электроэнергии (A_+) на основе обработанных временных рядов. Он включает применение моделей ARIMA, XGBoost и N-BEATS.

Основные функции модуля:

- прогнозирование потребления электроэнергии с использованием моделей ARIMA, XGBoost и N-BEATS;
- настройка гиперпараметров: порядок для модели ARIMA, глубина деревьев для модели XGBoost, архитектура нейронной сети для модели N-BEATS;
- временная кросс-валидация на горизонтах 168–720 часов;
- оценка по метрикам MAE и RMSE, с учётом вычислительной сложности и интерпретируемости.

При проектировании учитываются:

- адаптация гиперпараметров к вариабельности и длине рядов;

— баланс точности и вычислительных ресурсов.

Результатом является предоставление прогнозов для групп устройств с соответствующими метриками качества. Схема взаимодействия компонентов модуля изображена на рисунке 4.

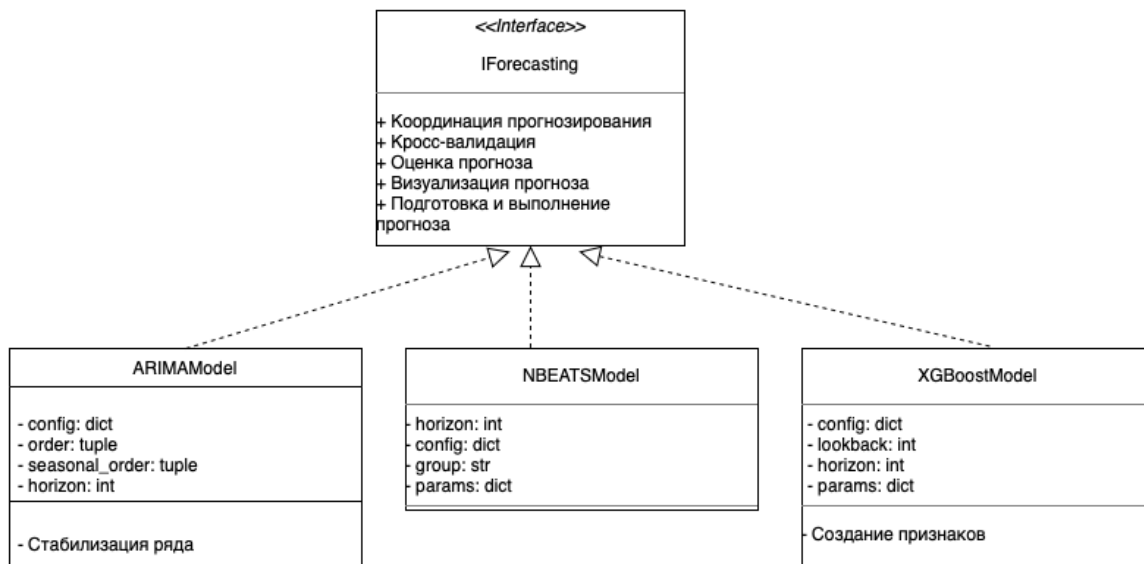


Рисунок 4 — Диаграмма классов модуля прогнозирования

3 Реализация приложения прогнозирования потребления электроэнергии

Реализация приложения для прогнозирования была выполнена с использованием языка программирования Python. Его выбор обусловлен наличием множества специализированных библиотек, таких как `statsmodels` [19], `xgboost` [20], `darts` [21] и `sklearn` [22], которые обеспечили эффективную обработку временных рядов, обучение моделей и анализ данных.

Проект реализован в модульной структуре, охватывающей этапы анализа, предобработки, прогнозирования и оценки качества. Все настраиваемые параметры моделей задаются в конфигурационном файле (листинг А.1, приложение А). Каждый этап сопровождается логированием и визуализацией; результаты сохраняются в каталог `graphics` для последующего анализа. Оценка качества прогноза проводится с использованием `sklearn.metrics`, вычисляя MAE и RMSE.

3.1 Реализация анализа данных

Анализ данных реализован в классе `TimeSeriesAnalyzer` с использованием библиотек `pandas`[24] для обработки данных, `matplotlib`[23] для визуализации и `statsmodels` для декомпозиции STL и выполнения теста Дики-Фуллера (ADF). Статистические показатели (среднее, медиана, стандартное отклонение, минимум, максимум) вычисляются через методы `pandas` и логируются для каждой группы (см. листинг 1).

Листинг 1: Вычисление статистик

```
1 stats = {'mean': df[self.target_column].mean(), 'std':  
    ↪ df[self.target_column].std(),  
2         'min': df[self.target_column].min(), 'max':  
    ↪ df[self.target_column].max(),  
3         'missing_ratio': df[self.target_column].isna().mean()}  
4 for key, value in stats.items():  
5     logging.info(f"{key}: {value}")
```

STL-разложение выполняется методом `plot_stl_decomposition` с использованием класса `STL` из модуля `statsmodels.tsa.seasonal` с параметром `robust=True` для устойчивости к выбросам. Компоненты (тренд, сезонность,

остатки) извлекаются и усредняются через функцию `pandas.Series.mean` (см. листинг 2).

Листинг 2: STL-разложение

```
1 stl = STL(group_data, period=self.period, robust=True)
2 result = stl.fit()
3 avg_trend = pd.Series(result.trend.dropna()).mean()
4 logging.info(f"Среднее тренда: {avg_trend}")
```

Тест стационарности `test_stationarity` использует метод `adfuller` модуля `statsmodels.tsa.stattools`. Метод `make_stationary` применяет дифференцирование через функцию `pandas.diff` до достижения стационарности (максимум второго порядка), удаляя пропуски с помощью функции `dropna` (см. листинг 3).

Листинг 3: Тест ADF и дифференцирование

```
1 result = adfuller(series)
2 p_value = result[1]
3 is_stationary = p_value < 0.05
4 diff_series = series.diff().dropna()
```

Графики автокорреляции (ACF) и частичной автокорреляции (PACF) строятся методом `plot_acf_pacf` с использованием функций `plot_acf` и `plot_pacf` из `statsmodels.graphics.tsaplots`, отображая до 720 лагов с выделением значимой для анализа сезонности (24, 168, 720) .

3.2 Реализация предобработки данных

Предобработка данных координируется классом `DataPrep`, интегрирующим специализированные модули: `MissingValueHandler` для заполнения пропусков, `AnomalyFix` для обработки аномалий и `NormFix` для нормализации.

3.2.1 Реализация заполнения пропусков

Класс `MissingValueHandler` обрабатывает пропуски с использованием библиотеки `pandas` для создания полного часового диапазона через метод `pd.date_range` и объединения данных с помощью метода `merge` (см. ли-

стинг 4). Метод заполнения (линейный, сезонный или KNN) определяется параметром конфигурации. Линейная интерполяция выполняется через функцию `pandas.interpolate`, сезонная — через усреднение прошлых периодов с помощью функции `pandas.mean`, а KNN использует класс `NearestNeighbors` из пакета `sklearn.neighbors` с преобразованием времени в секунды для вычисления евклидовых расстояний (см. листинг 5). Оставшиеся пропуски заполняются медианным или минимальным значением для дополнительных столбцов (`A_minus`, `R_plus`, `R_minus`) с использованием функции `pandas.fillna`.

Листинг 4: Создание полного часового диапазона

```
1 full_time_index = pd.date_range(start=min_time, end=max_time, freq='h')
2 full_df = pd.DataFrame({time_column: full_time_index, groupby: group})
3 merged_df = full_df.merge(group_data, on=[groupby, time_column],
  ↪ how='left')
```

Листинг 5: Выбор метода заполнения

```
1 if self.method == 'knn':
2     X_valid = (val_d[time_col] -
3     ↪ val_d[time_col].min()).dt.total_seconds().values.reshape(-1, 1)
4     knn = NearestNeighbors(n_neighbors=self.k, metric='euclidean')
5     knn.fit(X_valid)
6     distances, indices = knn.kneighbors(X_missing)
7 elif self.method == 'linear':
8     df[column] = df.groupby(groupby)[column].interpolate(method='linear')
9 elif self.method == 'seasonal':
10    df.loc[idx, column] = np.mean(seasonal_values)
```

3.2.2 Реализация обработки аномалий

Обработка аномалий реализована в классе `AnomalyFix` с использованием библиотек `pandas` для работы с временными рядами, `statsmodels` для STL-разложения, классом `RandomForestRegressor` пакета `sklearn.ensemble`. Поддерживаются методы поиска аномалий на основе STL-разложения, построения случайного леса, межквартильного размаха, а также возможность комбинирования методов, выбираемых на основе структуры временных рядов и задаваемых через конфигурацию.

Метод STL-разложения выявляет аномалии, анализируя остатки временного ряда. Основные шаги показаны в листинге 6.

Листинг 6: Обнаружение аномалий методом STL

```
1 stl = STL(series.dropna(), period=self.period, robust=True)
2 result = stl.fit()
3 residuals = result.resid
4 anomalies = np.abs(residuals) > threshold
```

Метод построения случайного леса использует регрессию с дополнительными признаками для обнаружения аномалий, как показано в листинге 7.

Листинг 7: Обнаружение аномалий методом RF

```
1 rf = RandomForestRegressor(n_estimators=self.n_estimators,
   ↪ random_state=42)
2 rf.fit(X_scaled, y)
3 y_pred = rf.predict(X_scaled)
4 anomaly_mask = np.abs(y - y_pred) > threshold
```

Метод определения аномалий на основе межквартильного размаха (IQR) проиллюстрирован в листинге 8.

Листинг 8: Обнаружение аномалий методом IQR

```
1 Q1 = group_data.quantile(0.25)
2 Q3 = group_data.quantile(0.75)
3 IQR = Q3 - Q1
4 anomaly_mask = (group_data < Q1 - 1.5 * IQR) | (group_data > Q3 + 1.5 *
   ↪ IQR)
```

Также возможно комбинирование методов, где при выраженной сезонности, определяемой через стандартное отклонение сезонной компоненты, применяется STL-разложение, в противном случае используется методом построения случайного леса, как показано в листинге 9.

Устранение аномалий реализовано в методе fix класса с параметром действия сглаживания или удаления, определяемым через конфигурацию.

Листинг 9: Комбинирование методов на основе сезонности

```
1 stl = STL(group_data, period=self.period, robust=True)
2 result = stl.fit()
3 seasonal_std = np.std(result.seasonal)
4 if seasonal_std > threshold:
5     anomalies = self._stl_anomaly_detection(group_data)
6 else:
7     anomalies = self._rf_anomaly_detection(df[df[group] == group_id],
      ↪ column, group, time_column)
```

Листинг 10: Исправление аномалий (интерполяция и удаление)

```
1 if action == 'interpolate':
2     interpolated = self._linear_interpolation(series, anomaly_positions,
      ↪ group_indices, df_copy, column, group, group_id)
3     df_copy.loc[orig_idx, column] = interpolated[pos]
4 else: # action == 'remove'
5     df_copy.loc[anomaly_indices, column] = np.nan
6     df_copy[column] = df_copy.groupby(group)[column].transform(lambda x:
      ↪ x.interpolate(method='linear'))
```

3.2.3 Реализация нормализации

Нормализация данных реализована в классе NormFix с использованием библиотек numpy для математических преобразований, pandas для работы с данными и json для сохранения параметров. Методы логарифмического преобразования и масштабирования к диапазону [0, 1], выбираемые через параметр method. Параметры нормализации определяются с учетом структуры данных и сохраняются в файл для последующего использования в обратном преобразовании.

Метод логарифмического преобразования применяется для положительных значений с добавлением 1, как показано в листинге 11.

Листинг 11: Логарифмическое преобразование

```
1 if (df_out[col] < 0).any():
2     raise ValueError("Отрицательные значения не допускаются для log")
3 df_out[col] = np.log1p(df_out[col])
4 self.params = {'method': 'log'}
```

Метод масштабирования к диапазону [0, 1] использует глобальные минимум и максимум для нормализации, как показано в листинге 12.

Листинг 12: Масштабирование в [0, 1]

```
1 x_min, x_max = df_out[col].min(), df_out[col].max()
2 df_out[col] = (df_out[col] - x_min) / (x_max - x_min + 1e-10) if x_max !=
  ↪ x_min else np.zeros_like(df_out[col])
3 self.params = {'method': 'minmax', 'min': float(x_min), 'max':
  ↪ float(x_max)}
```

Обратное преобразование реализовано в методе `denormalize`, где для логарифмического метода применяется экспонента, а для масштабирования выполняется обратное линейное преобразование, как показано в листинге 13.

Листинг 13: Обратное преобразование

```
1 if params['method'] == 'log':
2     df_out[col_name] = np.exp1(df_out[col_name])
3 else:
4     x_min, x_max = params['min'], params['max']
5     df_out[col_name] = df_out[col_name] * (x_max - x_min) + x_min
```

3.3 Реализация модели ARIMA

Реализация модели ARIMA выполнена в классе `ARIMAModel` с использованием пакета `statsmodels.tsa.arima.model` для прогнозирования временных рядов. Учитывая предполагаемую сезонность в данных, целесообразно расширить базовую модель ARIMA за счёт добавления сезонных гиперпараметров, что позволяет адаптировать её к периодическим шаблонам. Параметры модели оптимизируются перебором нескольких комбинаций гиперпараметров, отобранных исходя из проведённого анализа, что способствует нахождению более подходящих настроек для пользователя. Обучение и прогнозирование интегрированы с предварительной стабилизацией ряда, выполненной через `TimeSeriesAnalyzer`.

Обучение модели включает установку часовой частоты, стационарность ряда и настройку сезонных параметров, как показано в листинге 14.

Листинг 14: Обучение модели ARIMA

```
1 train_data = train_data.set_index('time_dt')
2 self.series = train_data['target']
3 self.series = self.series.asfreq('h').interpolate(method='linear')
4 diff_series, d = self.analyzer.make_stationary(self.series, max_diff=1)
5 self.series = diff_series
6 self.model = ARIMA(self.series, order=self.order,
   ↪ seasonal_order=self.seasonal_order)
7 self.model_fit = self.model.fit()
```

Прогноз генерируется на заданный горизонт с вычислением доверительных интервалов, как показано в листинге 15.

Листинг 15: Прогнозирование ARIMA

```
1 forecast_result = self.model_fit.forecast(steps=horizon)
2 forecast = np.array(forecast_result)
3 conf_int =
   ↪ self.model_fit.get_forecast(steps=horizon).conf_int(alpha=0.05)
4 if len(forecast) != horizon:
5     forecast = np.pad(forecast, (0, horizon - len(forecast)),
   ↪ mode='constant', constant_values=np.nan)
```

3.4 Реализация модели XGBoost

Реализация модели XGBoost выполнена в классе XGBoostModel с использованием библиотеки xgboost для прогнозирования временных рядов. Инициализация включает настройку гиперпараметров через конфигурацию. Признаки формируются методом `_create_features`, добавляя лаги (24, 48, 168 часов), час, день недели, месяц и индикатор выходного дня, с использованием функции `fillna`.

Обучение модели выполняется с использованием структуры `xgb.DMatrix` для оптимизации вычислений, с разделением данных на тренировочную и валидационную выборки в хронологическом порядке. Обучение проводится методом `xgb.train` с ранней остановкой, как показано в листинге 16.

Листинг 16: Обучение модели XGBoost

```
1 features_df = self._create_features(train_data)
2 X = features_df.drop(columns=['time_dt', 'target'])
3 y = features_df['target']
4 X_train, X_val = X.iloc[:-self.horizon], X.iloc[-self.horizon:]
5 y_train, y_val = y.iloc[:-self.horizon], y.iloc[-self.horizon:]
6 dtrain = xgb.DMatrix(X_train, label=y_train)
7 dval = xgb.DMatrix(X_val, label=y_val)
8 self.booster = xgb.train(
9     self.params,
10    dtrain,
11    num_boost_round=self.num_boost_round,
12    evals=[(dtrain, 'train'), (dval, 'val')],
13    early_stopping_rounds=self.early_stopping_rounds
14 )
```

Итеративное прогнозирование реализовано в методе `forecast` с рекурсивным обновлением признаков на каждом шаге, учитывая временные зависимости. Предсказанные значения используются для обновления лагов и временных признаков, как показано в листинге 17.

Листинг 17: Прогнозирование XGBoost

```
1 last_data = self._create_features(self.train_data.tail(self.lookback))
2 forecast_vals = []
3 current_data = last_data.copy()
4 for _ in range(horizon):
5     X = current_data.drop(columns=['time_dt', 'target']).tail(1)
6     dtest = xgb.DMatrix(X)
7     pred = self.booster.predict(dtest)[0]
8     forecast_vals.append(pred)
9     new_row = current_data.iloc[-1].copy()
10    new_row['time_dt'] += pd.Timedelta(hours=1)
11    new_row['target'] = pred
12    for lag in [24, 48, 168]:
13        new_row[f'lag_{lag}'] = forecast_vals[-1] if len(forecast_vals)
14        ↪ >= lag else current_data['target'].iloc[-lag]
15    new_row['hour'] = new_row['time_dt'].hour
16    new_row['day_of_week'] = new_row['time_dt'].dayofweek
17    new_row['is_weekend'] = 1 if new_row['time_dt'].dayofweek in [5, 6]
18    ↪ else 0
19    current_data = pd.concat([current_data, new_row.to_frame().T],
20    ↪ ignore_index=True)
```

3.5 Реализация модели N-BEATS

Реализация модели N-BEATS выполнена в классах `NBEATSInitializer` и `NBEATSWrapper` с использованием библиотеки `darts` и модели `NBEATSModel` для прогнозирования временных рядов. Инициализация параметров, таких как длина входных и выходных данных, количество блоков и слоев, а также эпох обучения, задаётся через конфигурацию в классе `NBEATSInitializer`. Модель создаётся в классе `NBEATSWrapper`, в качестве функции потерь выбирается `nn.MSELoss`. Данные преобразуются в объект `TimeSeries`.

Обучение модели реализовано в методе `train` класса `NBEATSWrapper`, где данные преобразуются и обучаются методом `fit`, как показано в листинге 18.

Листинг 18: Обучение модели N-BEATS

```
1 series = TimeSeries.from_dataframe(  
2     train_data, time_col='time_dt', value_cols='target',  
3     fill_missing_dates=True, freq='h'  
4 ).astype(np.float32)  
5 self.model.fit(series)
```

Прогнозирование реализовано в методе `forecast` с поддержкой рекурсивного подхода через метод `_recursive_forecast`, если заданный горизонт превышает величину `output_chunk_length`. Рекурсивный процесс последовательно предсказывает значения, используя предыдущие прогнозы как входные данные, как показано в листинге 19.

Листинг 19: Рекурсивное прогнозирование N-BEATS

```
1 forecast_vals = []  
2 steps_left = horizon  
3 while steps_left > 0:  
4     pred_horizon = min(self.horizon, steps_left)  
5     if not forecast_vals:  
6         forecast = self.model.predict(n=pred_horizon)  
7     else:  
8         current_series =  
9             ↪ TimeSeries.from_values(forecast_vals[-self.horizon:])  
10        forecast = self.model.predict(n=pred_horizon,  
11            ↪ past_values=current_series)  
12    forecast_vals.extend(forecast.values().flatten())  
13    steps_left -= pred_horizon
```

3.6 Реализация временного прогнозирования

Реализация прогнозирования временных рядов выполнена в модуле прогнозирования функциями прогнозирования `forecast` и вспомогательными утилитами для оценки и визуализации. Поддерживаются модели ARIMA, N-BEATS и XGBoost, инициализируемые через соответствующие классы (`ARIMAModel`, `NBEATSInitializer`, `XGBoostModel`). Прогнозирование, включая кросс-валидацию и финальный прогноз, выполняется для каждой группы с динамическим горизонтом.

Функция `cross_validation` обеспечивает временное разбиение данных для оценки моделей. Разбиение выполняется последовательно, как показано в листинге 20.

Листинг 20: Временная кросс-валидация

```
1 def cross_validation(df, group_col, target_col, folds=5, horizon=24):
2     series = df[target_col]
3     total_points = len(series)
4     if total_points < horizon * 2:
5         logging.warning(f"Мало данных: {total_points} точек")
6         return []
7     min_train = max(48, horizon * 2)
8     fold_size = (total_points - horizon) // max(1, (total_points -
9         ↪ horizon) // min_train)
10    folds = min(folds, fold_size)
11    if folds <= 0:
12        logging.warning("Недостаточно данных для фолдов")
13        return []
14    splits = [(df.iloc[:fold_size * k], df.iloc[fold_size * k:fold_size *
15        ↪ k + horizon])
16              for k in range(1, folds + 1) if fold_size * k >= min_train]
17    for i, (train_df, val_df) in enumerate(splits, 1):
18        logging.info(f"Фолд {i}: {len(train_df)} для обучения,
19            ↪ {len(val_df)} для проверки")
20    return splits
```

Прогнозирование и подготовка данных выполняются в функции `_prepare_and_forecast`, где модель генерирует прогнозы, а результаты денормализуются при необходимости. Визуализация сохраняется в виде графиков с помощью функции `plot_forecast`, а результаты и метрики экспортируются в файлы.

4 Тестирование системы прогнозирования потребления электроэнергии

Данный раздел охватывает результаты оценки предобработки данных и анализа точности моделей прогнозирования потребления активной электроэнергии (A_+ , с использованием кросс-валидации и финального прогноза. Основной целью являлось определение точности моделей по метрикам MAE и RMSE.

4.1 Анализ исходных данных

Анализ данных A_+ выполнен с применением класса `TimeSeriesAnalyzer` за период с 23 ноября 2023 года по 26 декабря 2024 года, охватывающий 402 дня (9646 часов) для 15 пользователей. Общие характеристики данных показывают диапазон значений A_+ от 0,00 до 3955,44 Вт·ч, со средним показателем 207,40 Вт·ч и стандартным отклонением 200,80 Вт·ч, что указывает на значительную вариабельность потребления. По группам средние значения варьируются от 106,77 Вт·ч до 417,43 Вт·ч, с максимальными значениями до 3955,44 Вт·ч и стандартным отклонением от 72,05 Вт·ч до 302,07 Вт·ч.

На рисунке 5 распределение A_+ демонстрирует правый скос с преобладанием значений ниже 500,00 Вт·ч и редкими пиками до 4000,00 Вт·ч, подчёркивая наличие аномалий.

Рисунок 6 отражает временные ряды для пяти групп, выявляя суточные колебания, пиковые значения до 2000,00 Вт·ч и наличие пропусков в данных, охватывающих целый месяц.

Рисунок 7 представляет STL-разложение для `uuid_ffb0f3ae`. Средние значения по всем пользователям, полученные из STL-разложения, составляют: тренд — 177,32 Вт·ч, сезонность — -0,04 Вт·ч, остатки — 30,12 Вт·ч. Тренд близок к общему среднему, указывая на стабильный уровень потребления, а минимальная сезонная компонента подтверждает слабую выраженность суточных циклов. Остатки 30,12 Вт·ч указывают на наличие аномалий, требующих обработки.

Таким образом, данные характеризуются высокой вариабельностью, аномалиями и пропусками в данных, что делает необходимым использование методов предобработки.

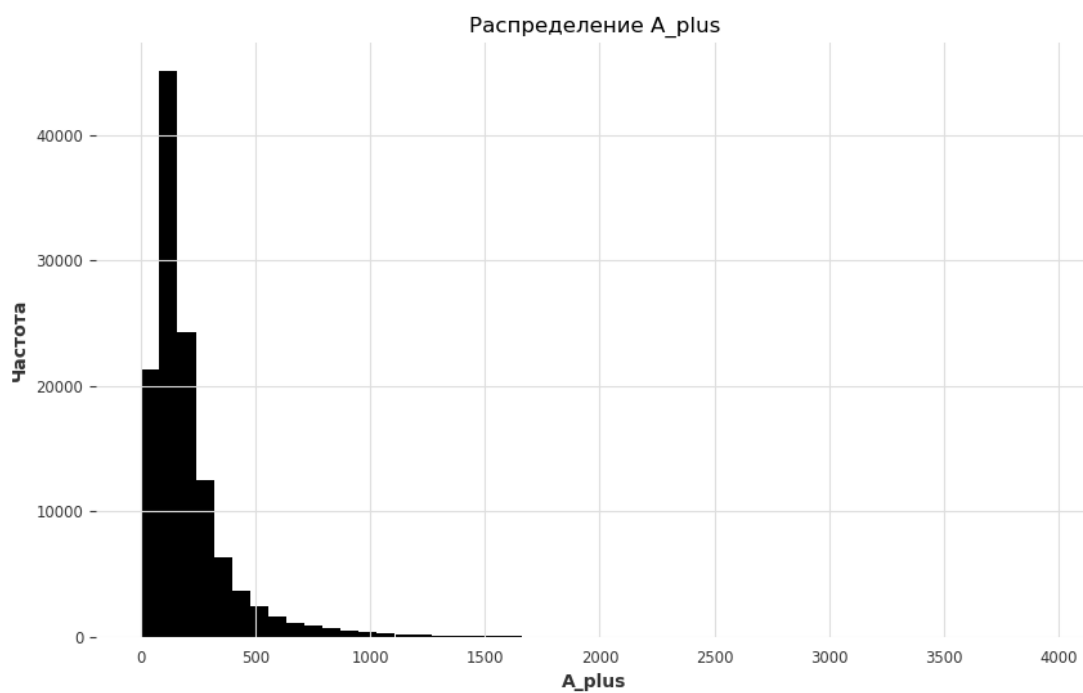


Рисунок 5 — Распределение A_{+} по всем группам

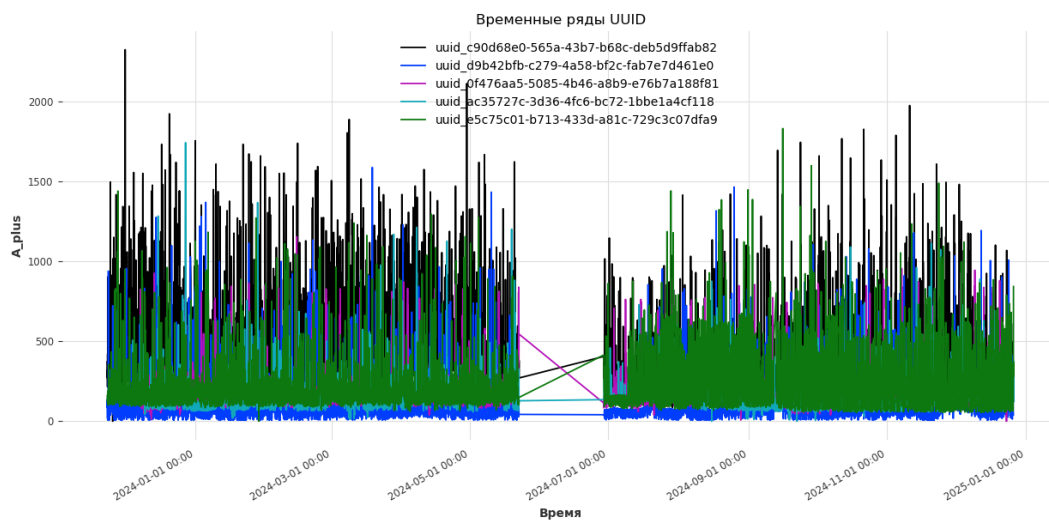


Рисунок 6 — Временные ряды A_{+} для пяти групп измерений

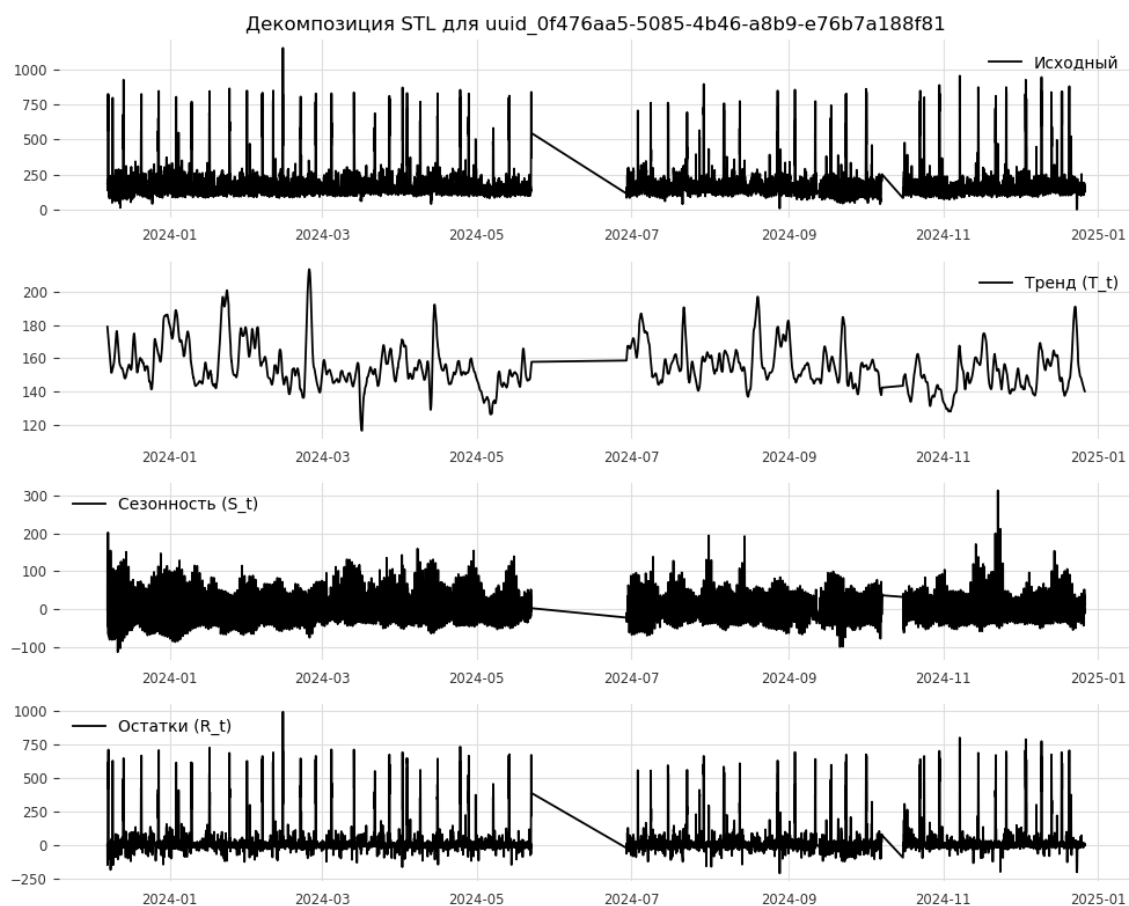


Рисунок 7 — STL-разложение для uuid_ffb0f3ae

4.2 Тестирование предобработки данных

Для подготовки данных к прогнозированию проведена предобработка. Исходный набор данных содержал 122545 записей с 20642 пропусками, что увеличило объем до 143187 записей после обработки. Дисперсия (σ^2) составила 39032,07, автокорреляция ($ACF(24)$) с лагом 24 — 0,37. Протестированы три комбинации методов: [knn, iqr, log], [knn, rf, log] и [linear, stl, linear]. Для KNN-интерполяции использовался параметр $k = 7$ (число ближайших соседей), выбранный на основе эмпирического анализа, обеспечивающего оптимальное заполнение пропусков с учетом локальной структуры данных и минимизации ошибки интерполяции. После обработки пропусков методом KNN значение среднего (μ) составило 205,72 Вт·ч, стандартного отклонения (σ) — 194,23 Вт·ч, автокорреляции — 0,43.

Комбинация методов [knn, iqr, log] применяет KNN-восстановления для заполнения пропусков, IQR для выявления 11119 аномалий и логарифмиче-

скую нормализацию. Выявление аномалий снизило значение среднего до 174,53 Вт·ч, стандартного отклонения — до 116,04 Вт·ч, максимума ($y_{\max}$) — до 820,29 Вт·ч, повысило автокорреляцию ($ACF(24)$) до 0,69. Нормализация увеличила автокорреляцию до 0,80, улучшив пригодность данных для моделей машинного обучения. Результаты приведены в таблице 1.

Таблица 1 — Статистика для комбинации методов [knn, iqr, log]

Этап	μ , Вт·ч	σ , Вт·ч	$y_{\max}$, Вт·ч	$ACF(24)$
Исходные	207,40	200,80	3955,44	0,37
После обработки пропусков	205,72	194,23	3955,44	0,43
После обработки аномалий	174,53	116,04	820,29	0,69
После нормализации	4,91	0,86	6,71	0,80

Далее протестирована комбинация методов [knn, rf, log], использующая иной подход к выявлению аномалий. Для метода построения случайного леса использованы параметры $n_estimators=250$ (число деревьев), обеспечивающее достаточную точность ансамбля при приемлемых вычислительных затратах, и $thresh=2,00$ (порог аномалий), оптимальный для выделения значительных выбросов без чрезмерного удаления данных. Метод обработки аномалий выявил 43134 аномалий, снизил значение среднего до 182,07 Вт·ч, стандартного отклонения — до 125,26 Вт·ч, максимума — до 1611,09 Вт·ч, повысил автокорреляцию до 0,72. Нормализация увеличила автокорреляцию до 0,85. Результаты отражены в таблице 2.

Таблица 2 — Статистика для комбинации методов [knn, rf, log]

Этап	μ , Вт·ч	σ , Вт·ч	$y_{\max}$, Вт·ч	$ACF(24)$
Исходные	207,40	200,80	3955,44	0,37
После обработки пропусков	205,72	194,23	3955,44	0,43
После обработки аномалий	182,07	125,26	1611,09	0,72
После нормализации	4,96	0,85	7,39	0,85

Комбинация методов [linear, stl, linear] применяет линейную интерполяцию для пропусков, STL-разложения для выявления 393 аномалий и линейную нормализацию. После обработки пропусков среднее составило 207,14 Вт·ч, стандартное отклонение — 197,57 Вт·ч, автокорреляция — 0,45. Выявление аномалий снизило значение среднего до 205,01 Вт·ч, стандартного отклонения — до 189,04 Вт·ч, максимума — до 3145,12 Вт·ч, повысило автокорреляцию до 0,48. Нормализация сохранила автокорреляцию на уровне 0,48. Результаты представлены в таблице 3.

Таблица 3 — Статистика для комбинации методов [linear, stl, linear]

Этап	μ , Вт·ч	σ , Вт·ч	$y_{\max}$, Вт·ч	ACF(24)
Исходные	207,40	200,80	3955,44	0,37
После обработки пропусков	207,14	197,57	3955,44	0,45
После обработки аномалий	205,01	189,04	3145,12	0,48
После нормализации	0,07	0,06	1,00	0,48

Предобработка снизила вариабельность данных и повысила автокорреляцию, улучшив их пригодность для моделирования. Комбинации методов [knn, iqr, log] и [knn, rf, log] обеспечили значительное снижение разброса и наивысшую автокорреляцию, что делает их оптимальными для машинного обучения. Комбинация методов [linear, stl, linear] показал меньшую эффективность из-за малого числа выявленных аномалий и низкой автокорреляции (0,48), сохраняющей значительную вариабельность.

4.3 Анализ стационарности временного ряда

Для настройки модели сезонной ARIMA проведена проверка стационарности временных рядов на предобработанных данных с использованием теста Дики-Фуллера (ADF). В основном ряды характеризуются стационарностью ($p\text{-value} < 0.05$), что указывает на стабильность дисперсии и отсутствие выраженных трендов, не требующих дополнительного дифференцирования.

Анализ автокорреляционной (ACF) и частичной автокорреляционной (PACF) функций выявил типичные шаблоны:

- умеренная сезонность с автокорреляцией — значимые лаги на 24 часа в ACF, резкий спад PACF после первого лага;
- выраженная суточная сезонность — четкие пики в ACF каждые 24 часа, два-три лага в PACF;
- суточная сезонность с трендом: суточные колебания в ACF, спад PACF после первого лага.

Перечисленные сценарии иллюстрируются на примерах (см. рисунки 8, 9, 10). Предлагаемые параметры сезонной ARIMA: $((0, 1, 2), (1, 0, 0, 24))$, $((2, 1, 1), (1, 0, 1, 24))$, $((1, 1, 2), (1, 0, 0, 24))$ соответственно.

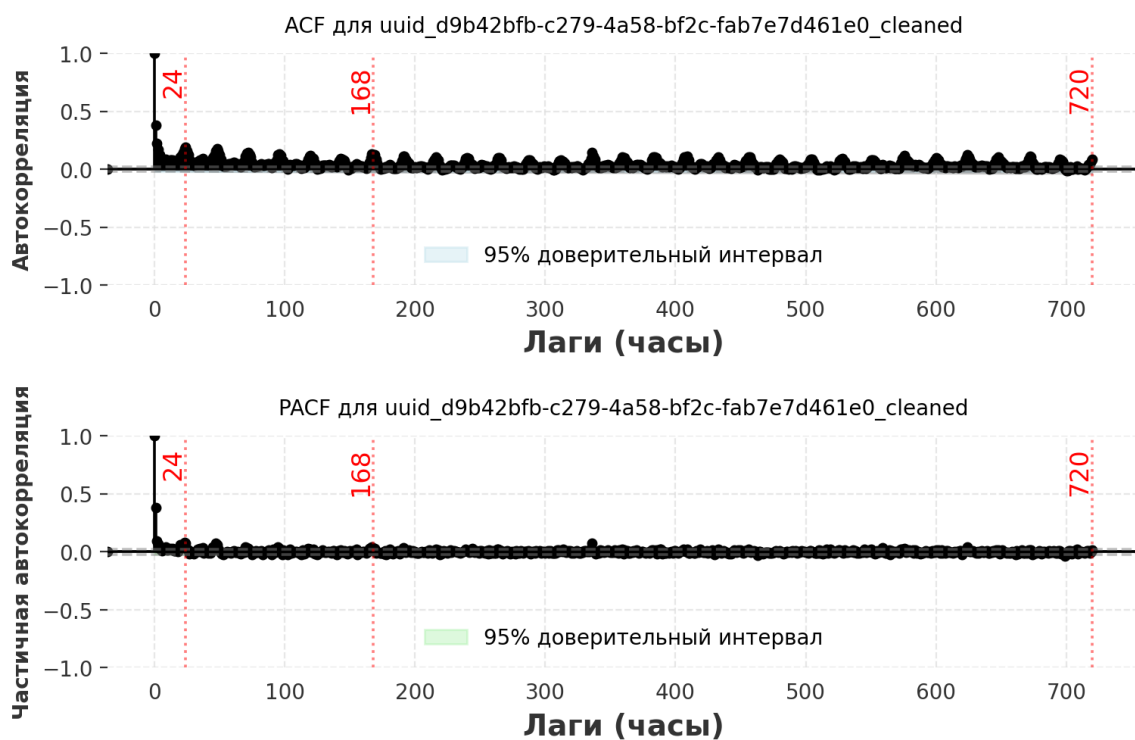


Рисунок 8 — Автокорреляция и частичная автокорреляция для типичного случая умеренной сезонности

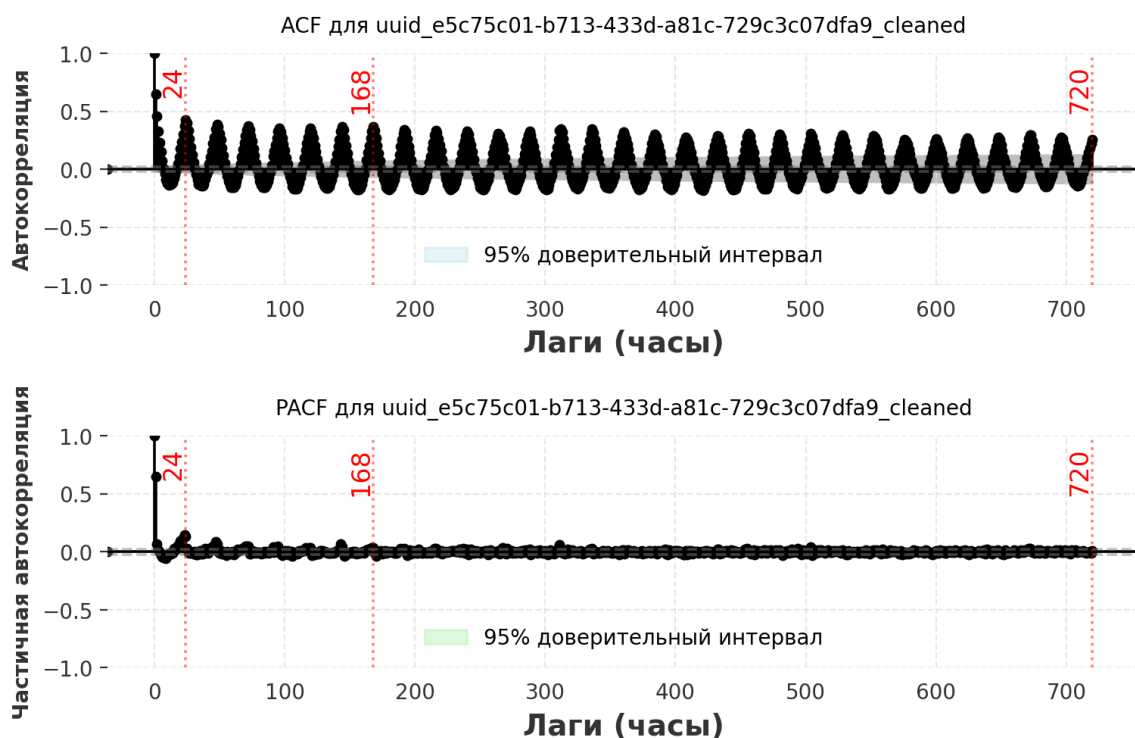


Рисунок 9 — Автокорреляция и частичная автокорреляция для типичного случая выраженной суточной сезонности

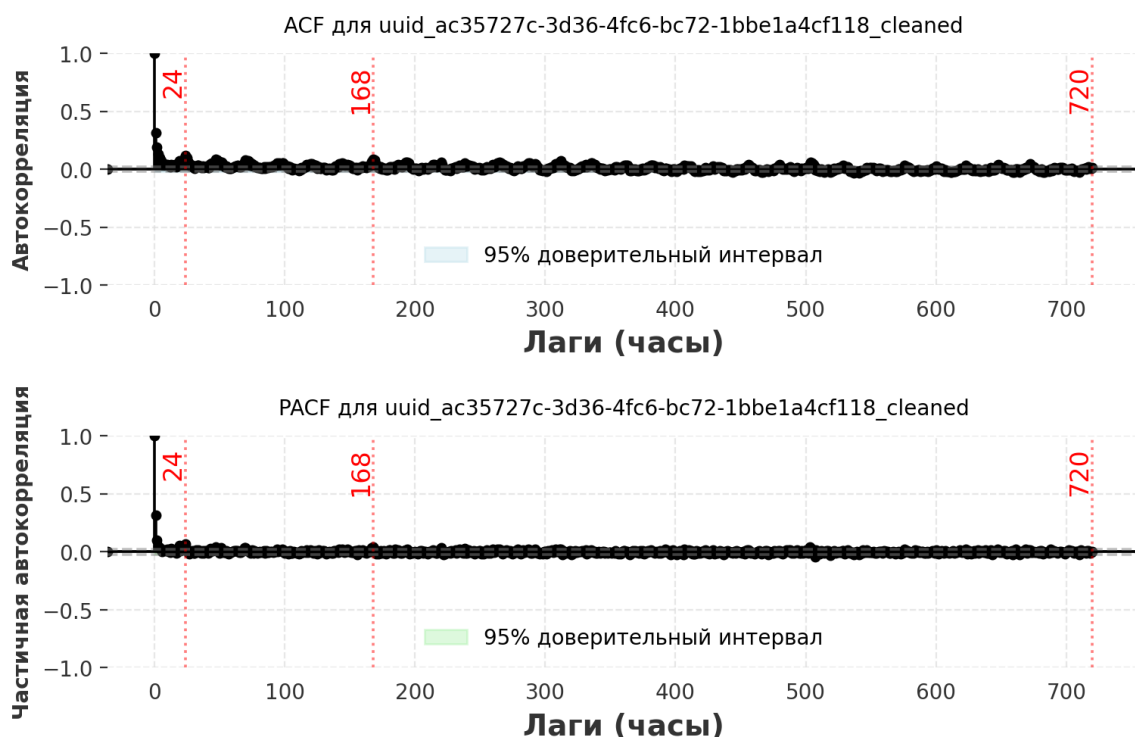


Рисунок 10 — Автокорреляция и частичная автокорреляция для типичного случая сезонности с трендом

4.4 Тестирование моделей прогнозирования

4.4.1 Тестирование модели сезонной ARIMA

Модель сезонной ARIMA была применена для прогнозирования потребления активной электроэнергии (A_+) на наборах данных с комбинациями методов предобработки [knn, iqr, log] и [knn, rf, log]. Анализ автокорреляции определил три комбинации гиперпараметров:

- $((0, 1, 2), (1, 0, 0, 24))$ — подходит для данных с умеренной сложностью;
- $((1, 1, 2), (1, 0, 0, 24))$ — подходит для данных с трендами;
- $((2, 1, 1), (1, 0, 1, 24))$ — подходит для сложных данных с сезонностью.

Оптимальные параметры выбирались по минимальному критерию Акаике (AIC) с сезонным периодом 24 часа. Предобработка включала KNN-восстановление для пропусков, методы межквартильного размаха или построения

случайного леса для устранения аномалий и логарифмическую нормализацию. Средние значения метрик представлены в таблице 4.

Таблица 4 — Метрики модели сезонной ARIMA для комбинации методов [knn, iqr, log] и [knn, rf, log]

Горизонт, часы	Кросс-валидация		Финальный прогноз	
	MAE, Вт·ч	RMSE, Вт·ч	MAE, Вт·ч	RMSE, Вт·ч
[knn, iqr, log]				
168	64,27	82,43	—	—
360	72,38	87,57	—	—
720	—	—	70,72	86,95
[knn, rf, log]				
168	64,94	84,58	—	—
360	76,96	93,83	—	—
720	—	—	75,32	94,26

Наибольшая ошибка отмечена для группы 100dce1b-a8a7 (например, MAE 254,39 Вт·ч, RMSE 283,21 Вт·ч на 360 часов для [knn, iqr, log]), наименьшая — для ac35727c-3d36 (MAE 27,37 Вт·ч, RMSE 29,44 Вт·ч на 168 часов). Аналогично для [knn, rf, log] — наибольшая ошибка у 100dce1b-a8a7, наименьшая у 96ce46ad-969d.

Пример прогноза модели сезонной ARIMA для группы 25e6de1d-9908, представленный на рисунке 11, иллюстрирует зависимости между предсказаниями и фактическими данными с учетом суточной сезонности.

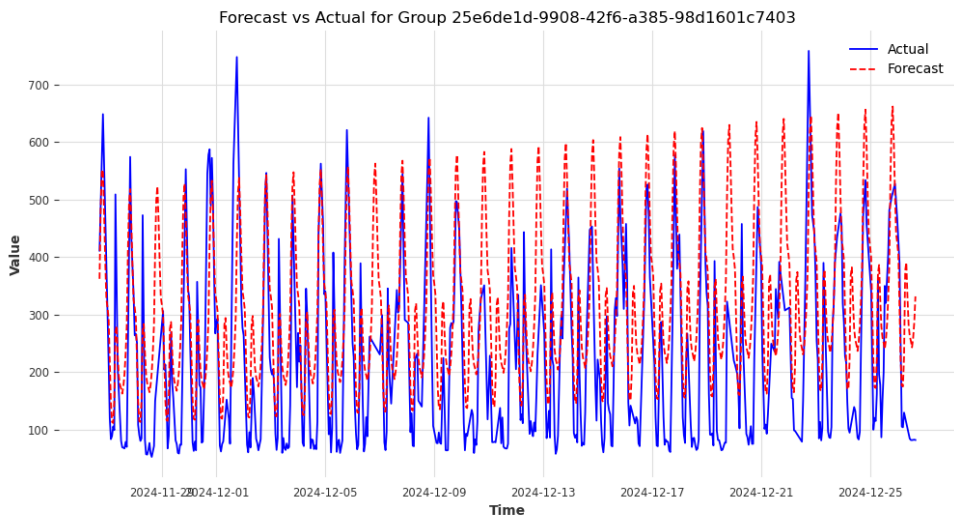


Рисунок 11 — Пример прогноза модели ARIMA

Модель показала схожую степень адаптации [knn, iqr, log] (MAE 64,27–72,38 Вт·ч, RMSE 82,43–87,57 Вт·ч) и [knn, rf, log] (MAE 64,94–76,96 Вт·ч, RMSE 84,58–94,26 Вт·ч). Высокие ошибки у 100dce1b-a8a7 указывают на то, что модель не полностью улавливает пиковые значения в данных, что требует дальнейшего анализа особенностей этого ряда.

4.4.2 Тестирование модели XGBoost

Модель XGBoost была применена для прогнозирования потребления активной электроэнергии (A_+) на наборах данных с комбинациями методов предобработки [knn, rf, log] и [knn, iqr, log]. Выбранные гиперпараметры ($num_boost_round = 300$, $max_depth = 10$, $learning_rate = 0,03$, $gamma = 0,005$, $lambda = 3$, $subsample = 0,87$, $colsample_bytree = 0,85$, $min_child_weight = 3$, $early_stopping_rounds = 20$) оптимизированы для улавливания суточной сезонности и высокой вариабельности данных.

Средние метрики для обеих комбинаций методов предобработки представлены в таблице 5. Анализ включал пользователей с наименьшими (ас35727с-3d36) и наибольшими (0f476aa5-5085) значениями ошибками.

Таблица 5 — Метрики модели XGBoost для комбинации методов [knn, rf, log] и [knn, iqr, log]

Горизонт, часы	Кросс-валидация		Финальный прогноз	
	MAE, Вт·ч	RMSE, Вт·ч	MAE, Вт·ч	RMSE, Вт·ч
[knn, rf, log]				
168	76,47	94,84	—	—
360	84,09	100,57	—	—
720	—	—	62,18	84,82
[knn, iqr, log]				
168	66,07	82,68	—	—
360	71,79	86,37	—	—
720	—	—	71,09	87,18

Для пользователя ас35727с-3d36 на комбинации методов [knn, iqr, log] ошибки минимальны: MAE 22,29 Вт·ч, RMSE 25,86 Вт·ч (168 часов) и MAE 12,18 Вт·ч, RMSE 18,32 Вт·ч (720 часов). Для пользователя 0f476aa5-5085 на [knn, rf, log] ошибки высоки: MAE 257,58 Вт·ч, RMSE 268,49 Вт·ч (168 часов), но улучшаются до MAE 32,35 Вт·ч, RMSE 57,51 Вт·ч (720 часов), тогда как

[knn, iqr, log] дает MAE 24,82 Вт·ч, RMSE 32,11 Вт·ч (168 часов) и MAE 27,44 Вт·ч, RMSE 35,11 Вт·ч (720 часов).

Пример прогноза модели для группы 25e6de1d-9908 представлен на рисунке 12.

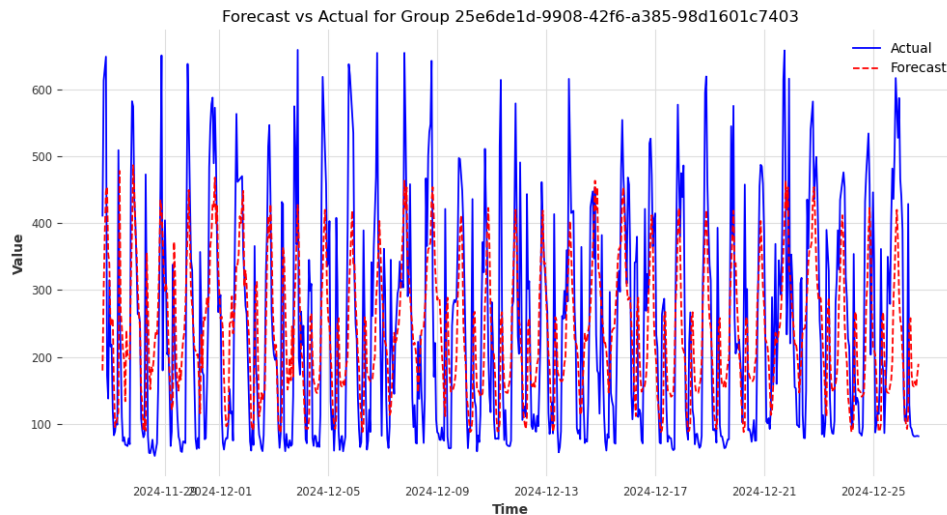


Рисунок 12 — Пример прогноза модели XGBoost

Модель XGBoost демонстрирует хорошую адаптацию в прогнозировании потребления активной электроэнергии, успешно обрабатывая суточную сезонность и вариабельность данных. В сравнении с моделью сезонной ARIMA, она лучше справляется с нелинейными шаблонами, хотя менее точно воспроизводит резкие пики, что особенно заметно для ряда 0f476aa5-5085 на наборе комбинации методов [knn, rf, log]. Комбинация [knn, iqr, log] обеспечивает более стабильные результаты, показывая важность выбора методов предобработки для минимизации сильных колебаний.

4.4.3 Тестирование модели N-BEATS

Модель N-BEATS демонстрирует высокую адаптацию в прогнозировании потребления активной электроэнергии (A_+), успешно улавливая суточную сезонность и сложные временные шаблоны. По сравнению с моделью сезонной ARIMA, она лучше моделирует нелинейные зависимости, хотя сталкивается с трудностями при воспроизведении резких пиков, особенно для ряда 0f476aa5-5085 на комбинации методов [knn, rf, log]. Выбранные гиперпараметры ($num_stacks = 2$, $num_blocks = 3$, $num_layers = 4$, $layer_widths = 512$, $n_epochs = 50$) опти-

мизированы для данных с высокой вариабельностью. Выбор значения параметра $num_stacks = 2$ обусловлен достаточностью двух уровней обработки для улавливания суточных и долгосрочных шаблонов, а значения параметра $num_blocks = 3$ обеспечивает баланс между гибкостью модели и вычислительной сложностью.

Средние значения метрик для комбинации методов предобработки [knn, rf, log] и [knn, iqr, log] представлены в таблице 6. Анализ включал пользователей с наименьшими (ас35727с-3d36) и наибольшими (0f476aa5-5085) ошибками.

Таблица 6 — Метрики модели N-BEATS для комбинации методов [knn, rf, log] и [knn, iqr, log]

Горизонт, часы	Кросс-валидация		Финальный прогноз	
	MAE, Вт·ч	RMSE, Вт·ч	MAE, Вт·ч	RMSE, Вт·ч
[knn, rf, log]				
168	70,00	88,56	—	—
360	79,10	95,57	—	—
720	—	—	57,17	78,89
[knn, iqr, log]				
168	62,03	81,14	—	—
360	70,02	85,81	—	—
720	—	—	59,05	76,67

Для пользователя ас35727с-3d36 на комбинации методов [knn, iqr, log] ошибки минимальны: MAE 24,86 Вт·ч, RMSE 27,63 Вт·ч (168 часов) и MAE 14,50 Вт·ч, RMSE 21,04 Вт·ч (720 часов). Для пользователя 0f476aa5-5085 на комбинации методов [knn, rf, log] ошибки высоки: MAE 211,72 Вт·ч, RMSE 221,76 Вт·ч (168 часов), но снижаются до MAE 40,55 Вт·ч, RMSE 66,43 Вт·ч (720 часов), тогда как комбинация [knn, iqr, log] дает MAE 28,22 Вт·ч, RMSE 35,30 Вт·ч (168 часов) и MAE 27,97 Вт·ч, RMSE 36,90 Вт·ч (720 часов).

Пример прогноза модели для группы 25e6de1d-9908 представлен на рисунке 13.

Модель успешно адаптируется к данным, особенно на комбинации методов [knn, iqr, log], обеспечивая более стабильные результаты. Однако для ряда 0f476aa5-5085 на комбинации [knn, rf, log] наблюдаются высокие ошибки, что указывает на недостаточную адаптацию к сильным колебаниям.

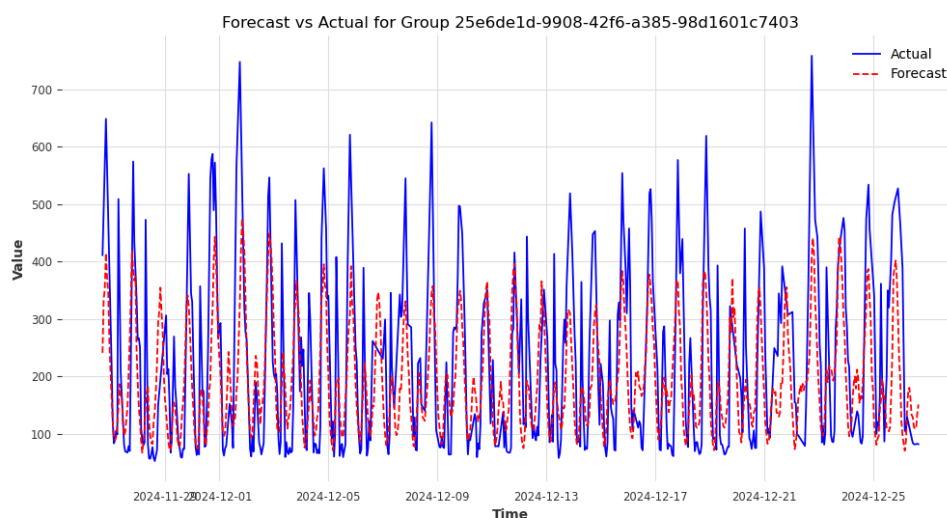


Рисунок 13 — Пример прогноза модели N-BEATS

4.5 Анализ результатов тестирования

Анализ исходных данных и результатов тестирования моделей ARIMA, N-BEATS и XGBoost выявил ключевые особенности прогнозирования потребления активной электроэнергии (A_+).

Исходный набор данных, охватывающий потребление электроэнергии 15 пользователей за 402 дня, характеризуется высокой вариабельностью, наличием аномалий, пропусков и суточной сезонностью. Проведенная предобработка с использованием комбинаций методов [knn, iqr, log] и [knn, rf, log] оказалась наиболее эффективной, обеспечивая лучшую пригодность данных для моделирования за счет усиления автокорреляции и устранения выбросов. Комбинация методов [linear, stl, linear] показала низкую эффективность, что подтверждает ее ограниченную применимость для данных с выраженной нерегулярностью.

Сравнение моделей показало, что модель N-BEATS демонстрирует наилучшую способность улавливать сложные нелинейные зависимости и сезонные шаблоны, особенно на данных, обработанных комбинацией методов [knn, iqr, log]. Модель ARIMA, несмотря на хорошую интерпретируемость и устойчивость, уступает в обработке нерегулярных пиков. Модель XGBoost эффективен для данных с четкой сезонностью, но менее стабилен при сильных колебаниях. Наиболее точные прогнозы достигнуты для пользователей с низкой вариабельностью потребления, тогда как для рядов с аномальными пиками ошибки прогнозирования остаются

значительными, что указывает на необходимость дальнейшей оптимизации предобработки.

Полученные результаты подтверждают важность тщательной предобработки данных и выбора модели прогнозирования, адаптированной к специфике данных, для достижения надежных прогнозов потребления электроэнергии.

ЗАКЛЮЧЕНИЕ

В рамках работы разработано приложение для прогнозирования потребления активной электроэнергии (A_+) на основе данных IoT, включающее модули анализа, предобработки и прогнозирования значений временных рядов. Проведён анализ характеристик данных, реализованы методы предобработки данных и применены различные подходы к прогнозированию, включая статистические модели (сезонная ARIMA) и методы машинного обучения (модели XGBoost и N-BEATS). Тестирование выполнено с использованием временной кросс-валидации и метрик MAE и RMSE.

Сравнительный анализ показал, что методы машинного обучения эффективнее улавливают нелинейные зависимости и сложные шаблоны в данных по сравнению со статистическими подходами, которые лучше справляются с устойчивыми сезонными рядами. Предобработка данных существенно повысила их качество, однако обработка нерегулярных аномалий остаётся ограничением, требующим дальнейшей оптимизации.

В качестве направлений развития разработанного приложения можно рассматривать расширение набора данных за счет дополнительных параметров, например, характеризующих погодные условия, а также рассмотрение гибридных моделей прогнозирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1.Осипьенок И. В. и Гришин И. В. [и др.] Интеллектуальный энергосетевой Smart Grid // Модели, системы, сети в экономике, технике, природе и обществе. — 2017. — № 21. — С. 109–116.
- 2.Box G. E. P., Jenkins G. M., Reinsel G. C. Time Series Analysis: Forecasting and Control. — 5th ed. — Hoboken, NJ: Wiley, 2015. — 712 p. — ISBN 978-1-118-67502-1.
- 3.Cleveland R. B. et al. STL: A Seasonal-Trend Decomposition Procedure Based on Loess // Journal of Official Statistics. — 1990. — Vol. 6, No. 1. — p. 3–73. — ISSN 0282-423X.
- 4.Dickey-Fuller and Augmented Dickey-Fuller Tests // ATSA-ES. — Электронный ресурс. — URL: <https://atsa-es.github.io/atsa-labs/sec-boxjenkins-aug-dickey-fuller.html> (дата обращения: 2 июня 2025 г.).
- 5.Little R. J. A., Rubin D. B. Statistical Analysis with Missing Data. — 3rd ed. — Hoboken, NJ: Wiley, 2019. — 462 p. — ISBN 978-0-470-52979-9.
- 6.Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. — 2nd ed. — Australia: OTexts, 2018. — 504 p. — ISBN 978-0-9875071-1-2.
- 7.Yang Y., Darmont J., Ravat F., Teste O. Dimensional Data KNN-Based Imputation // Proceedings of the 26th European Conference on Advances in Databases and Information Systems (ADBIS 2022). — 2022. — P. 315–329. — DOI: 10.1007/978-3-031-15740-0_23. — (Lecture Notes in Computer Science).
- 8.Chandola V. et al. Anomaly Detection: A Survey // ACM Computing Surveys. — 2009. — Vol. 41, No. 3. — p. 1–58. — DOI: 10.1145/1541880.1541882. — ISSN 0360-0300.
- 9.Tukey J. W. Exploratory Data Analysis. — Reading, MA: Addison-Wesley, 1977. — 688 p. — ISBN 978-0-201-07616-6.
- 10.Breiman L. Random Forests // Machine Learning. — 2001. — Vol. 45, No. 1. — p. 5–32. — DOI: 10.1023/A:1010933404324. — ISSN 0885-6125.
- 11.Hastie T. et al. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. — 2nd ed. — New York: Springer, 2009. — 745 p. — ISBN 978-0-387-84857-0.

12. Shumway R. H., Stoffer D. S. Time Series Analysis and Its Applications: With R Examples. — 4th ed. — Cham: Springer, 2017. — 562 p. — ISBN 978-3-319-52451-1.
13. Akaike H. A New Look at the Statistical Model Identification // IEEE Transactions on Automatic Control. — 1974. — Vol. 19, No. 6. — p. 716–723. — DOI: 10.1109/TAC.1974.1100705. — ISSN 0018-9286.
14. Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. — San Francisco, CA, 2016. — p. 785–794. — DOI: 10.1145/2939672.2939785.
15. Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine // The Annals of Statistics. — 2001. — Vol. 29, No. 5. — p. 1189–1232. — DOI: 10.1214/aos/1013203451. — ISSN 0090-5364.
16. Oreshkin B. N. et al. N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting // International Conference on Learning Representations (ICLR 2020). — Virtual Event, 2020.
17. ServiceNow Research. N-BEATS: Neural-Network Based Model for Univariate Timeseries Forecasting. — Электронный ресурс. — URL: <https://github.com/ServiceNow/N-BEATS> (дата обращения: 2 июня 2025 г.).
18. Brownlee J. Deep Learning for Time Series Forecasting: Predict the Future with MLPs, CNNs and LSTMs in Python. — Ebook. — Machine Learning Mastery, 2021. — 287 p. — ISBN 978-1-64763-398-2.
19. Seabold S. et al. statsmodels: Statistical Computations and Models for Python. — Электронный ресурс. — 2010–2025. — URL: <https://www.statsmodels.org/stable/index.html> (дата обращения: 2 июня 2025 г.).
20. Chen T. et al. XGBoost: eXtreme Gradient Boosting. — Электронный ресурс. — 2016–2025. — URL: <https://xgboost.readthedocs.io/en/stable/> (дата обращения: 2 июня 2025 г.).
21. Darts: Time Series Made Easy in Python // Unit8. — Электронный ресурс. — URL: <https://unit8co.github.io/darts/> (дата обращения: 25 мая 2025 г.).

- 22.scikit-learn: Machine Learning in Python // scikit-learn Developers. — Электронный ресурс. — URL: <https://scikit-learn.org/stable/> (дата обращения: 25 мая 2025 г.).
- 23.Matplotlib: Python Plotting // Matplotlib Development Team. — Электронный ресурс. — URL: <https://matplotlib.org/stable/> (дата обращения: 25 мая 2025 г.).
- 24.Pandas: Python Data Analysis Library // The Pandas Development Team. — Электронный ресурс. — URL: <https://pandas.pydata.org/> (дата обращения: 25 мая 2025 г.).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлен конфигурационный файл, используемый в работе для настройки параметров обработки данных и прогнозирования.

Листинг А.1: Конфигурационный файл

```
1 data:
2   raw: ../data/raw/file20.csv
3   processed: ../data/processed/
4       clean_data_knn_rf_minmax.csv
5   norm_params: data/processed/norm_params.json
6   forecasts: data/forecasts/forecasts.csv
7   column_mapping:
8       time_dt: time_dt
9       group: uuid
10      target: A_plus
11 preprocessing:
12     target_column: A_plus
13     period: 24
14     miss_method: knn
15     anom_method: stl
16     anom_act: mixed
17     norm_method: minmax
18     k: 4
19     thresh: 4.0
20     rf_n_estimators: 150
21     lookback: 168
22     process_missing: true
23     process_anomalies: true
24     process_normalization: true
25 forecasting:
26     model: xgboost #   arima nbeats xgboost
27     horizon: 168
28     horizons: [72, 96, 120, 144, 168]
29     arima:
30         order: [2, 0, 2]
31         auto: true
32         s: 24
33     nbeats:
```

```
33     num_stacks: 2
34     num_blocks: 3
35     num_layers: 4
36     layer_widths: 512
37     n_epochs: 5
38 xgboost:
39     n_estimators: 200
40     max_depth: 6
41     learning_rate: 0.05
42     gamma: 0
43     lambda: 1
44 graphics:
45     output_dir: graphics
46 logging:
47     log_file: logs/project.log
```